Chương 4: Factory Pattern & Abstract Factory Pattern (Phần 1)



Factory Pattern – Hãy sẵn sàng "tiếp đón" một vài thiết kế OO "kết hợp lỏng lẻo"

Có nhiều thứ để tạo ra đối tượng thay vì sử dụng toán tử ***new***. Với **Factory Pattern** bạn sẽ học được rằng khởi tạo là một hoạt động không phải lúc nào cũng được thực hiện công khai, và điều đó thường dẫn đến các vấn đề về sự phụ thuộc. Và bạn không muốn điều đó, phải không? Tìm hiểu làm thế nào **Factory Pattern** có thể cứu bạn khỏi sự phụ thuộc giữa các lớp.



Khi bạn thấy từ khóa "new", hãy nghĩ về các lớp con.

Vâng, khi bạn sử dụng ***new***, bạn chắc chắn đang tạo một object của lớp con, do đó, chắc chắn là một "programing to implement" (Khái niệm này được nhắc ở Chương 1 (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html)), không phải là "programing to interface" . Và nó là một câu hỏi hay; bạn đã học được rằng việc gắn code của bạn vào một lớp cấp thấp có thể dễ vỡ và kém linh hoạt hơn.

```
Duck duck = new MallardDuck();
```

We want to use interfaces to keep code flexible.

But we have to create an instance of a concrete class!

Khi bạn có cả một tập hợp các lớp con liên quan, thường thì bạn sẽ buộc phải viết code như thế này:

```
Duck duck;

if (picnic) {
    duck = new MallardDuck();
} else if (hunting) {
    duck = new DecoyDuck();
} else if (inBathTub) {
    duck = new RubberDuck();
}
```



We have a bunch of different duck classes, and we don't know until runtime which one we need to instantiate.

Ở đây, chúng tôi đã có một số lớp cụ thể được khởi tạo và quyết định khởi tạo được đưa ra trong thời gian chạy (runtime) tùy thuộc vào một số điều kiện.

Khi bạn thấy code thế này, bạn biết rằng khi đến lúc phải thay đổi hoặc mở rộng, bạn sẽ phải mở lại code này và kiểm tra những gì cần thêm (hoặc xóa). Thông thường loại code thế này kết thúc trong một số phần của ứng dụng khiến việc bảo trì và cập nhật trở nên khó khăn hơn và dễ bị lỗi hơn.

Nhưng bạn phải tạo một đối tượng và Java chỉ cung cấp cho chúng ta một cách duy nhất để tạo một đối tượng (new), phải không? Vì vậy, ta sẽ làm gì? Có điều gì sai khi sử dụng "new"?

Về mặt kỹ thuật, không có gì sai với **new** cả, nó là một phần cơ bản của Java. Thủ phạm thực sự là người bạn cũ của chúng ta – **SỰ THAY ĐỔI,** và **SỰ THAY ĐỔI** ảnh hưởng đến việc sử dụng **new** của chúng ta.

Bằng cách "code với một interface" (programing to interface), bạn biết rằng bạn có thể tự bảo vệ mình khỏi rất nhiều thay đổi có thể xảy ra với một hệ thống. Tại sao? Nếu code của bạn được dựa trên một interface, thì nó sẽ hoạt động với bất kỳ lớp mới nào implement interface đó thông qua đa hình. Tuy nhiên, khi bạn có code dựa trên lớp con, bạn sẽ gặp rắc rối vì code đó có thể phải thay đổi khi các lớp con mới được thêm vào. Vì vậy, nói cách khác, lớp của bạn sẽ không bị "đóng cho việc sửa đổi" (vi phạm nguyên tắc đã đề cập ở Chương 3: Decorator Pattern – Đối tượng trang trí (https://toihocdesignpattern.com/chuong-3-head-first-design-patterns-tieng-viet-decorator-pattern-doi-tuong-trang-tri.html)). Để có thể mở rộng nó với các loại lớp con mới, bạn sẽ bắt buộc phải "mở" lại nó.

Vậy bạn có thể làm gì? Đây là lúc bạn có thể dựa vào Nguyên tắc thiết kế OO để tìm kiếm manh mối. Hãy nhớ rằng, nguyên tắc đầu tiên của chúng ta (Strategy (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html)) liên quan đến sự thay đổi và hướng dẫn chúng ta xác định các khía cạnh khác nhau và tách chúng khỏi những gì vẫn giữ nguyên.

Sử dụng sức mạnh bộ não

Làm thế nào bạn có thể lấy tất cả các đoạn code "khởi tạo lớp con" trong ứng dụng của bạn và tách hoặc đóng gói chúng khỏi những phần còn lại?

Xác định các khía cạnh khác nhau

Hãy nói rằng bạn có một cửa hàng pizza và là chủ cửa hàng pizza tiên tiến ở Objectville, bạn có thể sẽ viết một số dòng code như thế này:

```
Pizza orderPizza() {
        Pizza pizza = new Pizza();

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();
        return pizza;
}
```

For flexibility, we really want this to be an abstract class or interface, but we can't directly instantiate either of those.

NHƯNG BẠN CẦN NHIỀU LOẠI PIZZA HƠN...

Vì vậy, sau đó bạn thêm một số dòng code định nghĩa loại pizza phù hợp và sau đó khởi tạo bánh pizza:

```
Pizza orderPizza(String type) {
        Pizza pizza;

        if (type.equals("cheese")) {
            pizza = new CheesePizza();
        } else if (type.equals("greek") {
            pizza = new GreekPizza();
        } else if (type.equals("pepperoni") {
            pizza = new PepperoniPizza();
        }

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();
        return pizza;
}
```

We're now passing in the type of pizza to orderPizza.

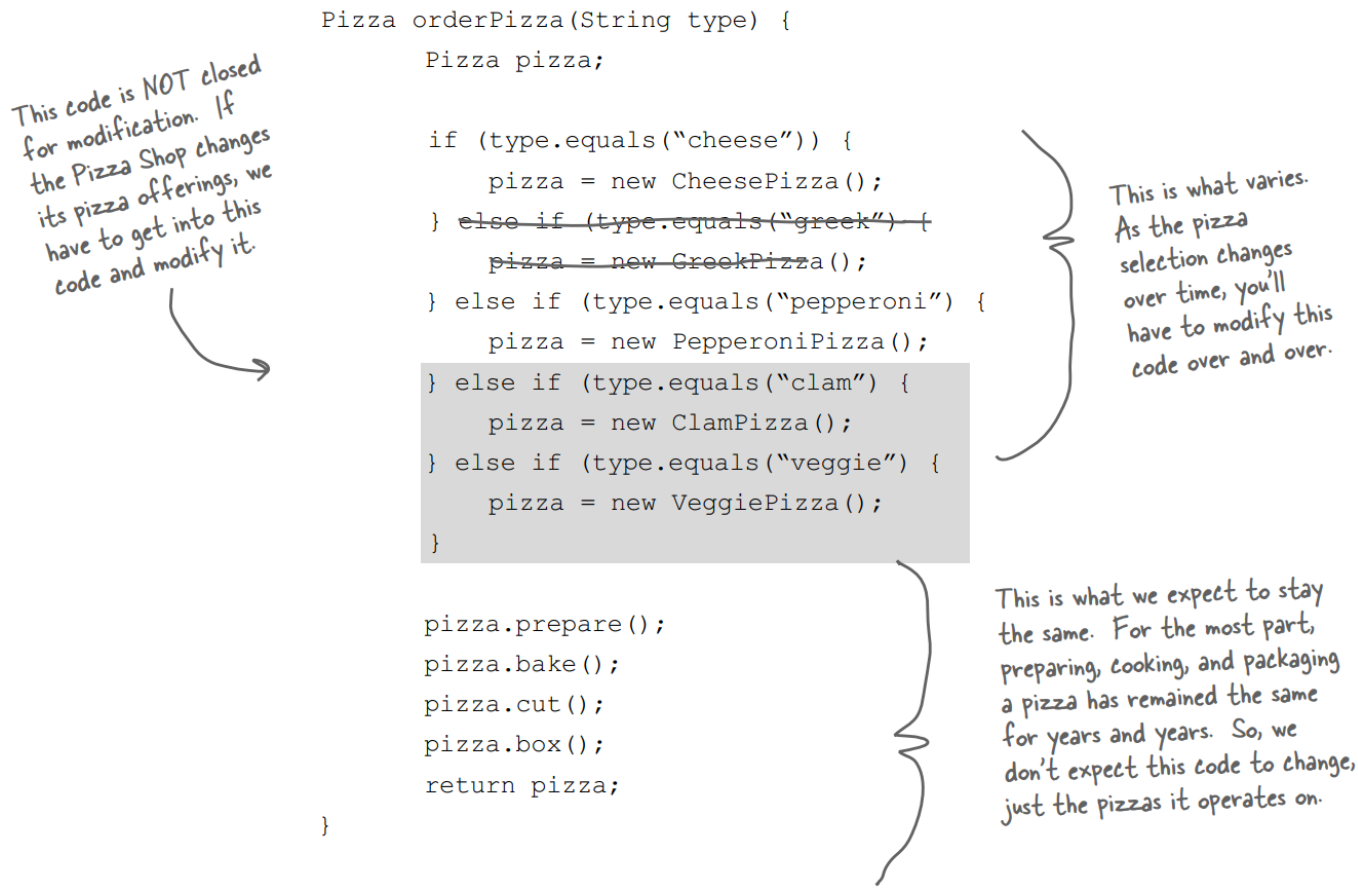
Based on the type of pizza, we instantiate the correct concrete class and assign it to the pizza instance variable. Note that each pizza here has to implement the Pizza interface.

Once we have a Pizza, we prepare it (you know, roll the dough, put on the sauce and add the toppings & cheese), then we bake it, cut it and box it!

Each Pizza subtype (CheesePizza, VeggiePizza, etc.) knows how to prepare itself.

Nhưng phải thêm nhiều loại pizza nữa

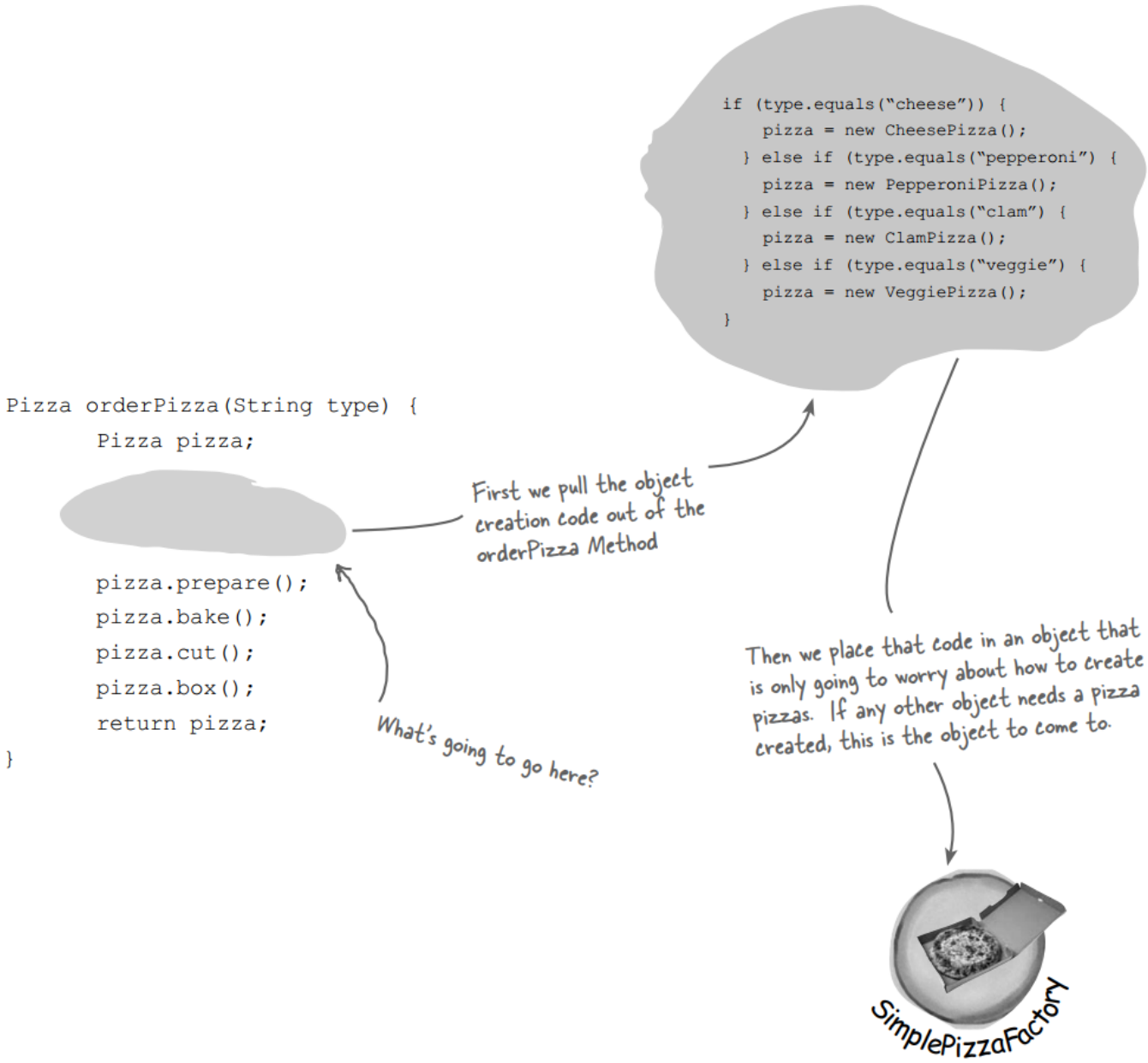
Bạn nhận ra rằng tất cả các đối thủ cạnh tranh của bạn đã thêm một vài loại pizza "hợp thời trang" vào thực đơn của họ: **Pizza Clam** và **Veggie Pizza**. Rõ ràng bạn cần theo kịp đối thủ, vì vậy bạn sẽ **thêm** các mục này vào thực đơn của mình. Và gần đây bạn không bán được nhiều bánh **Greek Pizzas**, vì vậy bạn quyết định **loại bỏ** nó ra khỏi thực đơn:



Rõ ràng, việc xử lý khởi tạo trên thực sự gây rối cho phương thức ***orderPizza()*** của chúng ta và ngăn không cho nó "đóng cho việc sửa đổi". Nhưng bây giờ chúng ta đã biết cái gì đang thay đổi và cái gì không, có lẽ đã đến lúc đóng gói nó.

Đóng gói việc khởi tạo đối tượng

Vì vậy, bây giờ chúng ta biết rằng sẽ tốt hơn khi chuyển việc tạo đối tượng ra khỏi phương thức ***orderPizza()***. Nhưng bằng cách nào? Chà, những gì chúng tôi sẽ làm là lấy phần code tạo pizza và chuyển nó ra một đối tượng khác chỉ liên quan đến việc tạo ra pizza.



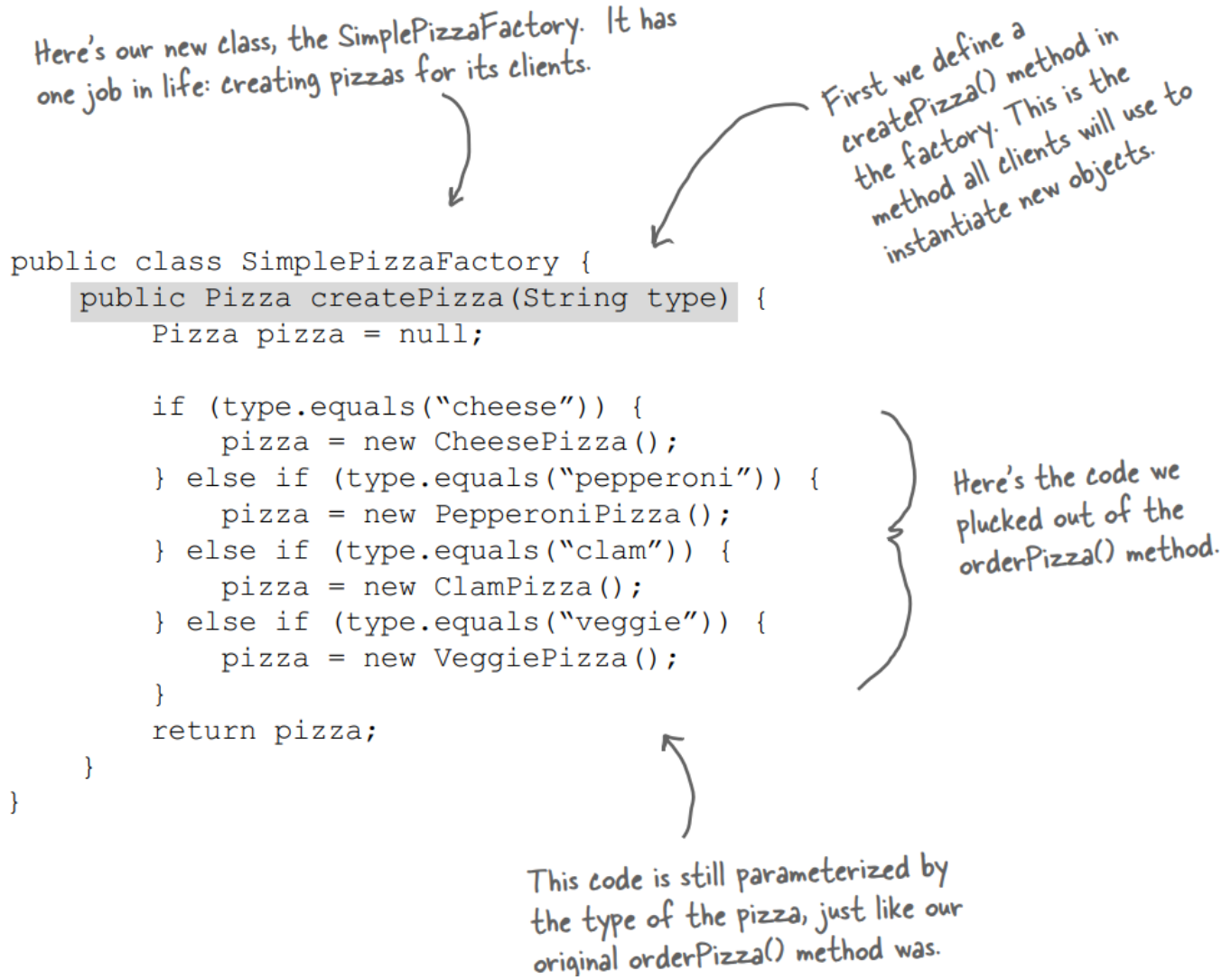
Chúng tôi đã đặt tên cho đối tượng mới này: chúng tôi gọi nó là Factory.

Các nhà máy Factory xử lý phần chi tiết của việc tạo đối tượng. Khi chúng ta có một **SimplePizzaFactory**, phương thức ***orderPizza()*** của chúng ta sẽ trở thành một máy khách (client) của đối tượng đó. Bất cứ khi nào nó cần một chiếc bánh pizza, nó sẽ yêu cầu nhà máy sản xuất pizza làm một cái. Đã qua rồi cái thời mà phương thức ***orderPizza()*** cần biết về loại pizza **Greek Pizzas** hay là **Clam pizzas**. Bây giờ phương thức ***orderPizza()*** chỉ quan tâm rằng nó nhận được một chiếc bánh pizza đang implement giao diện Pizza để nó có thể gọi ***prepare()***, ***bake()***, ***cut()***, và ***box()***.

Chúng tôi vẫn có một vài chi tiết để điền vào đây; chẳng hạn, phương thức ***orderPizza()*** thay thế code tạo của nó bằng gì? Hãy implement một nhà máy đơn giản cho cửa hàng pizza và tìm hiểu …

Xây dựng một Pizza Factory đơn giản

Chúng tôi sẽ bắt đầu với chính nhà máy. Những gì chúng tôi sẽ làm là định nghĩa một lớp gói gọn việc tạo đối tượng cho tất cả các loại pizza. Nó đây…



Không có câu hỏi nào ngớ ngẩn

Hỏi: Lợi ích của việc này là gì? Có vẻ như chúng ta chỉ đang đẩy vấn đề sang một đối tượng khác.

Trả lời: Một điều cần nhớ là **SimplePizzaFactory** có thể có nhiều client. Chúng ta chỉ thấy phương thức ***orderPizza()*** gọi đến nó; tuy nhiên, có thể có một lớp client khác – **PizzaShopMenu** cũng sử dụng factory để tạo pizza và lấy mô tả và giá của nó. Chúng ta cũng có thể có một lớp **HomeDelivery** gọi đến nhà máy tạo pizza theo cách khác với lớp **PizzaShop**, nhưng tất cả chúng đều là client của **Factory**.

Vì vậy, bằng cách đặt việc khởi tạo pizza trong nhiều lớp, giờ đây chúng ta chỉ có một lớp để thực hiện sửa đổi khi cần thiết. Đừng quên, chúng ta cũng sắp xóa việc khởi tạo lớp con khỏi phương thức ***orderPizza()*** !

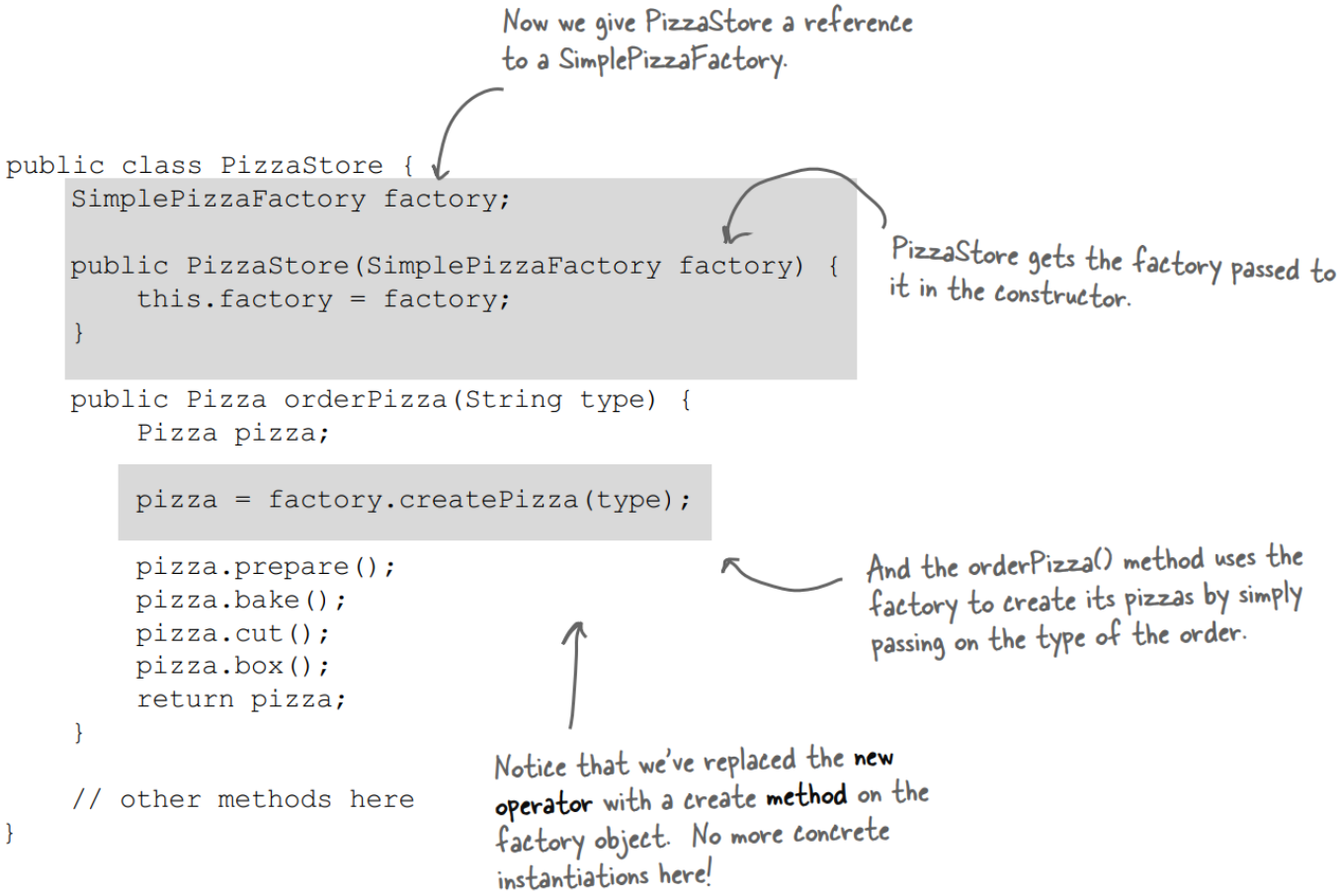
Hỏi: Tôi đã thấy một thiết kế tương tự, trong đó một factory sẽ được định nghĩa là một phương thức static. Có sự khác biệt gì không?

Trả lời: Định nghĩa factory là một phương thức static là một kỹ thuật phổ biến và thường được gọi là một **"static factory"**. Tại sao sử dụng phương thức tĩnh ư? Bởi vì bạn không cần khởi tạo một đối tượng để sử dụng phương thức factory. Nhưng hãy nhớ rằng nó cũng có sự bất lợi là bạn không thể có lớp con và thay đổi hành vi của phương thức *get*, *set*.

Làm lại lớp PizzaStore với Simple Factory

Bây giờ là thời gian để sửa code. Những gì chúng tôi muốn làm là dựa vào factory để tạo ra các loại pizza. Dưới đây là những thay đổi:

Bây giờ lớp PizzaStore (client) sẽ tham chiếu đến **SimplePizzaFactory** (factory) của chúng ta.




```
public class PizzaStore {
    SimplePizzaFactory factory;

    public PizzaStore(SimplePizzaFactory factory) {
        this.factory = factory;
    }

    public Pizza orderPizza(String type) {
        Pizza pizza;

        pizza = factory.createPizza(type);

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();
        return pizza;
    }

    // other methods here
}
```

Sử dụng sức mạnh bộ não

Chúng ta biết rằng việc kết hợp các object (composition) thay vì implementation cho phép chúng ta thay đổi hành vi một cách linh hoạt trong runtime bởi vì chúng có thể thay đổi giữa trong và ngoài việc kế thừa. Làm thế nào chúng ta có thể sử dụng điều này trong PizzaStore? Những triển khai factory nào chúng ta có thể trao đổi giữa trong và ngoài?

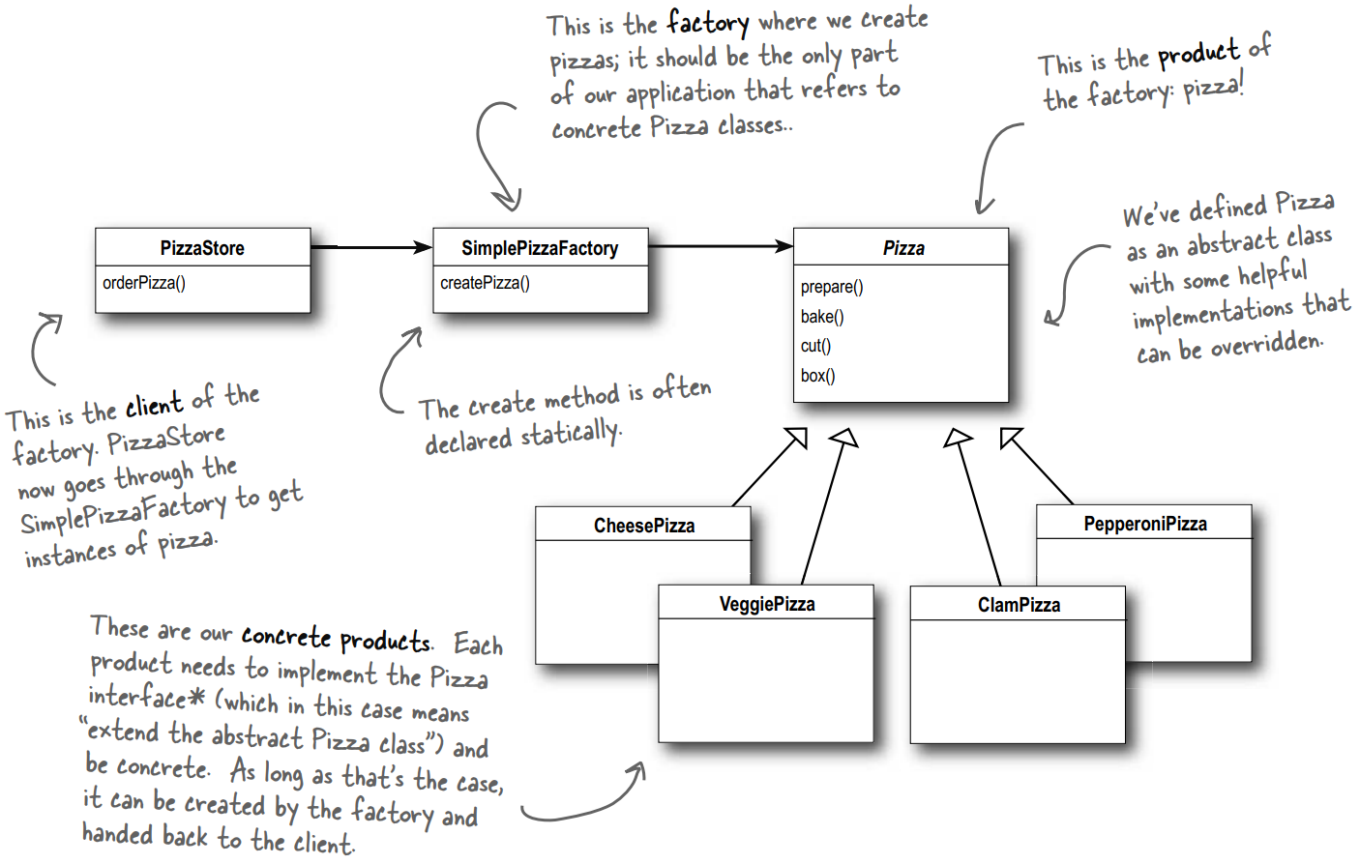
Định nghĩa Simple Factory

Simple Factory không thực sự là một mẫu thiết kế; nó là một programming idiom (thành ngữ lập trình). Nhưng nó thường được sử dụng, vì vậy chúng tôi sẽ tặng cho nó một danh hiệu "Head First Pattern Honorable Mention".



Một số nhà phát triển đã nhầm với "Factory Pattern" (những gì chúng ta tìm hiểu nãy giờ không phải là Factory Pattern), vì vậy lần sau, khi giữa bạn và một developer khác không biết nói gì, bạn đã có một chủ đề hay để bàn luận.

Simple Factory không phải là một mẫu THỰC SỰ, không có nghĩa là chúng ta không nên xem xét cách mà nó kết hợp với nhau. Hãy cùng xem sơ đồ lớp của Cửa hàng Pizza mới của chúng tôi:



Hãy nghĩ về Simple Factory như một sự khởi động. Phía sau, chúng ta cũng sẽ khám phá Abstract Factory Pattern (https://toihocdesignpattern.com/chuong-4-factory-pattern-va-abstract-factory-pattern-phan-2.html). Nhưng đừng lo, vẫn còn nhiều điều để nói ở đây!

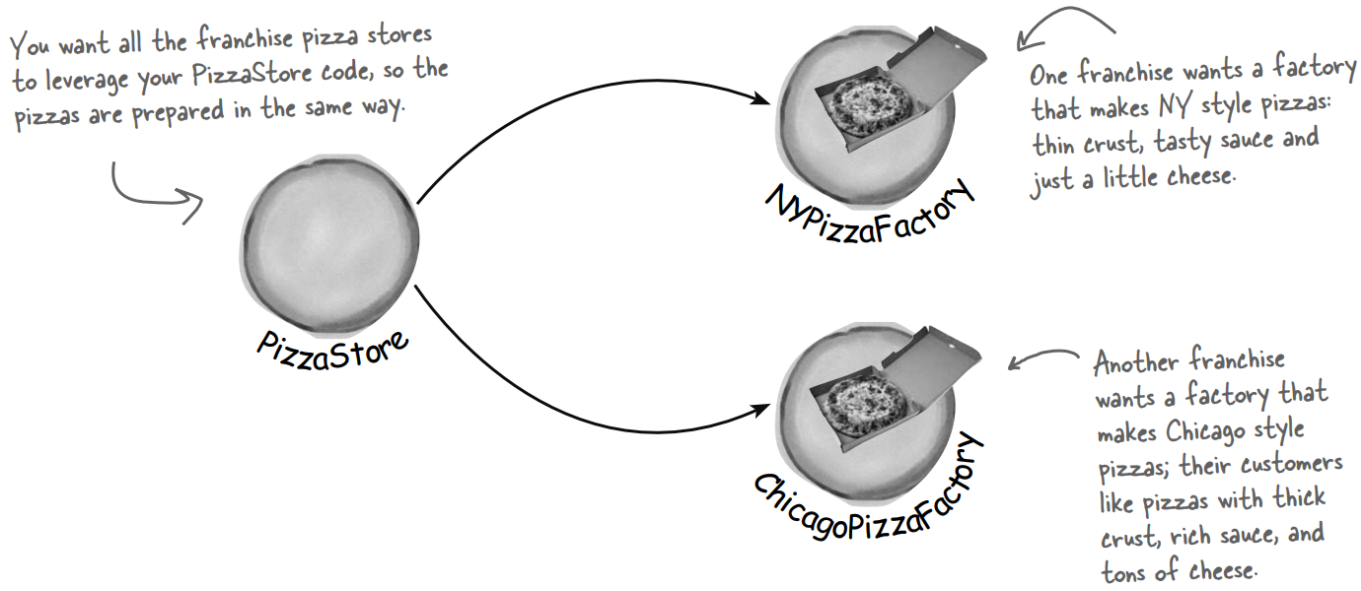
** Chỉ là một lời nhắc khác: trong các mẫu thiết kế, cụm từ "implement an interface", không phải lúc nào cũng có nghĩa là "tạo một lớp implement một Java interface, bằng cách sử dụng từ khóa "implements" trong khai báo lớp con". Một lớp cụ thể implements một phương thức từ một supertype (có thể là một lớp HOẶC interface) vẫn được coi là một "implement an interface" của supertype đó.*

Nhượng quyền cửa hàng pizza

(Nhượng quyền thương hiệu hiểu nôm na là cho phép dùng thương hiệu của bạn để mở một chi nhánh khác, và chia lại lợi nhuận cho bạn. Royal tea – Trà sữa ở tất cả các nơi không phải cùng 1 người sở hữu, đó đã được nhượng quyền lại cho người khác)

Objectville PizzaStore của bạn đã làm rất tốt đến nỗi bạn đã vượt qua đối thủ và bây giờ mọi người đều muốn có một PizzaStore ở nơi họ sống. Là người nhượng quyền (franchiser), bạn muốn đảm bảo chất lượng của hoạt động nhượng quyền và vì vậy bạn muốn họ sử dụng lại code (mà bạn bạn đã bỏ thời gian ra test) để các cửa hàng được nhượng quyền sử dụng.

Nhưng có sự khác biệt về khu vực? Mỗi nơi được nhượng quyền có thể muốn cung cấp các loại pizza khác nhau (New York, Chicago và California,…), tùy thuộc vào vị trí cửa hàng được nhượng quyền và thị hiếu của những người sành pizza ở địa phương.



CHÚNG TÔI ĐÃ THẤY MỘT CÁCH TIẾP CẬN …

Nếu chúng tôi loại bỏ lớp **SimplePizzaFactory** và tạo ra ba lớp factory khác: **NYPizzaFactory**, **ChicagoPizzaFactory** và **CaliforniaPizzaFactory**, thì chúng tôi có thể kết hợp **PizzaStore** với một factory phù hợp và nhượng quyền thương mại là điều tốt. Đó là một cách tiếp cận.

Hãy xem điều này sẽ trông như …

```
NYPizzaFactory nyFactory = new NYPizzaFactory();
PizzaStore nyStore = new PizzaStore(nyFactory);
nyStore.order("Veggie");
```

Here we create a factory for making NY style pizzas.

Then we create a PizzaStore and pass it a reference to the NY factory.

...and when we make pizzas, we get NY-styled pizzas.

```
ChicagoPizzaFactory chicagoFactory = new ChicagoPizzaFactory();
PizzaStore chicagoStore = new PizzaStore(chicagoFactory);
chicagoStore.order("Veggie");
```

Likewise for the Chicago pizza stores: we create a factory for Chicago pizzas and create a store that is composed with a Chicago factory. When we make pizzas, we get the Chicago flavored ones

NHƯNG BẠN MUỐN KIỂM SOÁT CHẤT LƯỢNG HƠN MỘT CHÚT ...

Vì vậy, bạn đã test qua lớp **SimpleFactory** và những gì bạn nhận thấy là những nơi được nhượng quyền đang sử dụng factory của bạn để tạo ra pizza, nhưng họ bắt đầu sử dụng các cách riêng cho phần còn lại của quy trình làm bánh pizza: họ sẽ nướng mọi thứ một cách khác biệt, họ quên cắt pizza và họ sử dụng hộp đựng bánh của bên thứ ba.



Xem xét lại vấn đề một chút, bạn thấy rằng những gì bạn thực sự muốn làm là tạo ra một khuôn khổ gắn kết cửa hàng và việc tạo ra pizza với nhau, nhưng vẫn cho phép mọi thứ vẫn linh hoạt.

Trong code trước đây của chúng tôi, trước **SimplePizzaFactory**, chúng tôi đã có code làm bánh pizza gắn liền với **PizzaStore**, nhưng nó không linh hoạt. Vì vậy, làm thế nào chúng ta có thể có bánh pizza của chúng tôi?

Một Framework cho cửa hàng pizza

Có một cách để "bản địa hóa" tất cả các hoạt động làm bánh pizza cho lớp PizzaStore, nhưng vẫn cho phép nhượng quyền tự do theo style của từng khu vực riêng.

Những gì chúng tôi sẽ làm là đưa phương thức **createPizza()** trở lại vào **PizzaStore**, nhưng lần này là một phương thức trừu tượng (abstract), sau đó tạo một lớp con **PizzaStore** cho mỗi kiểu vùng.

Đầu tiên, hãy xem các thay đổi của PizzaStore:

PizzaStore is now abstract (see why below).

```
public abstract class PizzaStore {

    public Pizza orderPizza(String type) {
        Pizza pizza;

        pizza = createPizza(type);

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();

        return pizza;
    }

    abstract Pizza createPizza(String type);
}
```

Now createPizza is back to being a call to a method in the PizzaStore rather than on a factory object.

All this looks just the same...

Now we've moved our factory object to this method.

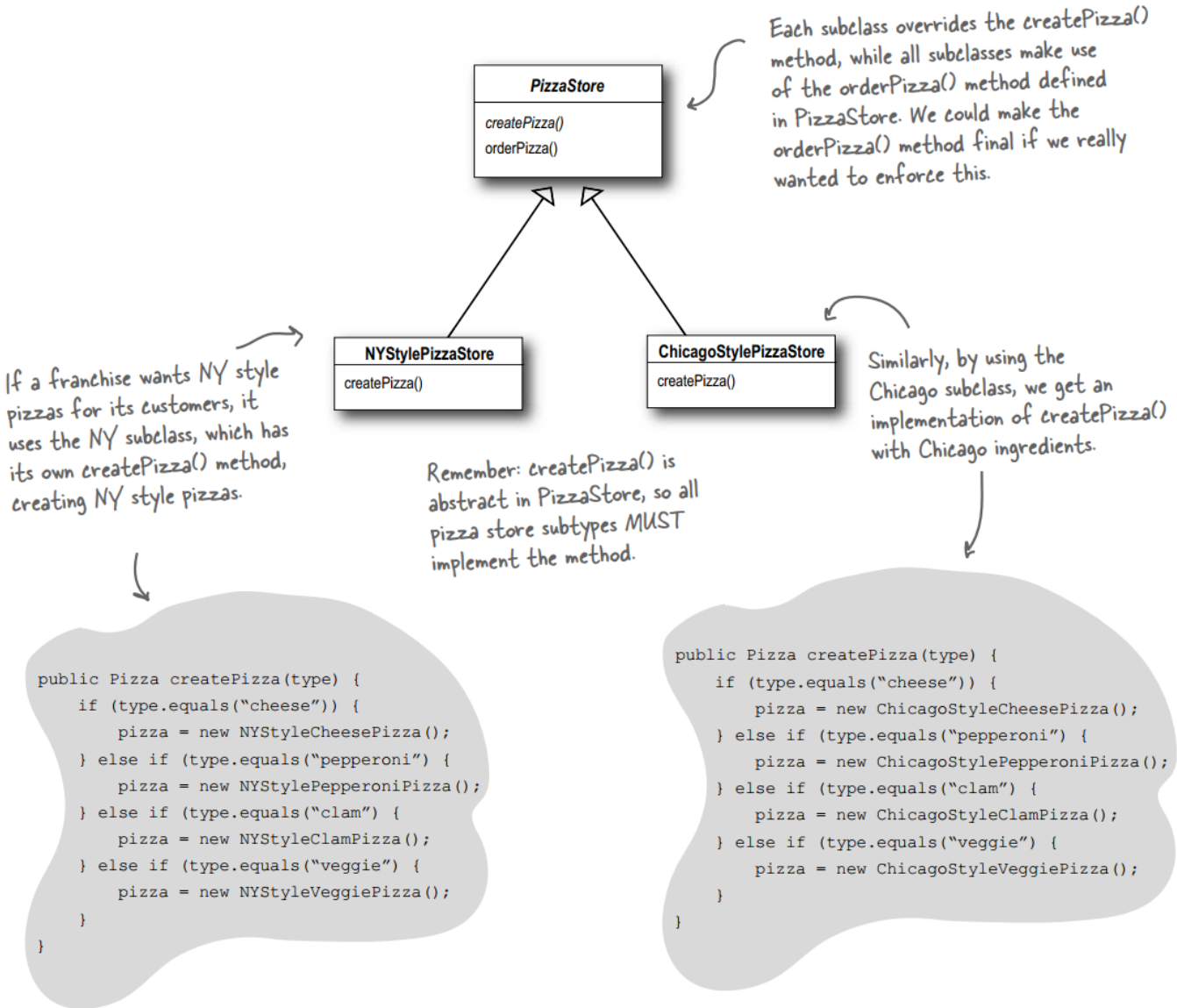
Our "factory method" is now abstract in PizzaStore.

Bây giờ chúng tôi đã có một cửa hàng đang chờ các lớp con kế thừa; chúng tôi sẽ có các lớp con cho từng loại khu vực (**NYPizzaStore**, **ChicagoPizzaStore**, **CaliforniaPizzaStore**) và mỗi lớp con sẽ đưa ra quyết định về những gì tạo nên một chiếc bánh pizza. Hãy xem điều này sẽ hoạt động như thế nào.

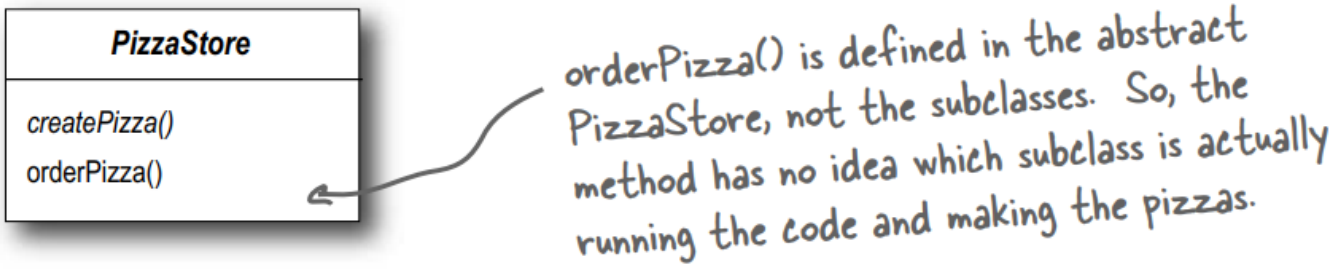
Cho phép lớp con quyết định

Hãy nhớ rằng, PizzaStore đã có một hệ thống đặt hàng được test kỹ lưỡng trong phương thức **orderPizza()** và bạn muốn đảm bảo rằng nó phù hợp với tất cả các nơi được nhượng quyền.

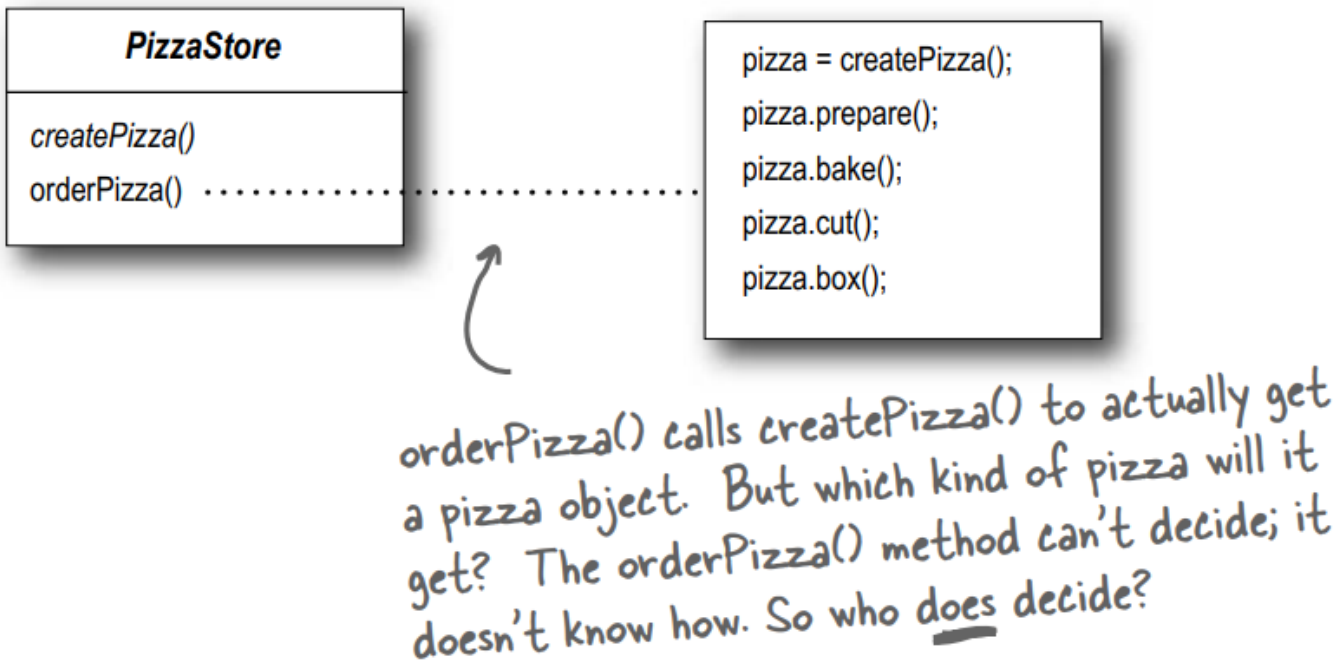
Điều khác biệt giữa các **PizzaStores** ở từng địa phương là phong cách pizza họ làm – Pizza New York có lớp vỏ mỏng, Chicago Pizza dày, v.v. – và chúng tôi sẽ đưa tất cả các biến thể này vào phương thức **createPizza()** và nó sẽ chịu trách nhiệm tạo ra đúng loại pizza. Chúng ta làm điều này là bằng cách cho phép mỗi lớp con của PizzaStore định nghĩa lại phương thức **createPizza()**. Vì vậy, chúng ta sẽ có một số lớp con cụ thể của **PizzaStore**, mỗi lớp có các biến thể pizza riêng, tất cả đều nằm trong khung **PizzaStore** và vẫn sử dụng phương thức **orderPizza()**.



Chà, hãy nghĩ về nó theo quan điểm của phương thức **orderPizza()** của lớp **PizzaStore**: nó được định nghĩa trong lớp trừu tượng **PizzaStore**, nhưng các loại cụ thể chỉ được tạo trong các lớp con.



Bây giờ, để hiểu thêm một chút, phương thức **orderPizza()** thực hiện rất nhiều thứ với một đối tượng Pizza (như chuẩn bị, nướng, cắt, đóng hộp), nhưng bởi vì Pizza là abstract, **orderPizza()** không biết lớp con thực sự là gì, có liên quan như thế nào. Nói cách khác, nó đã tách ra!



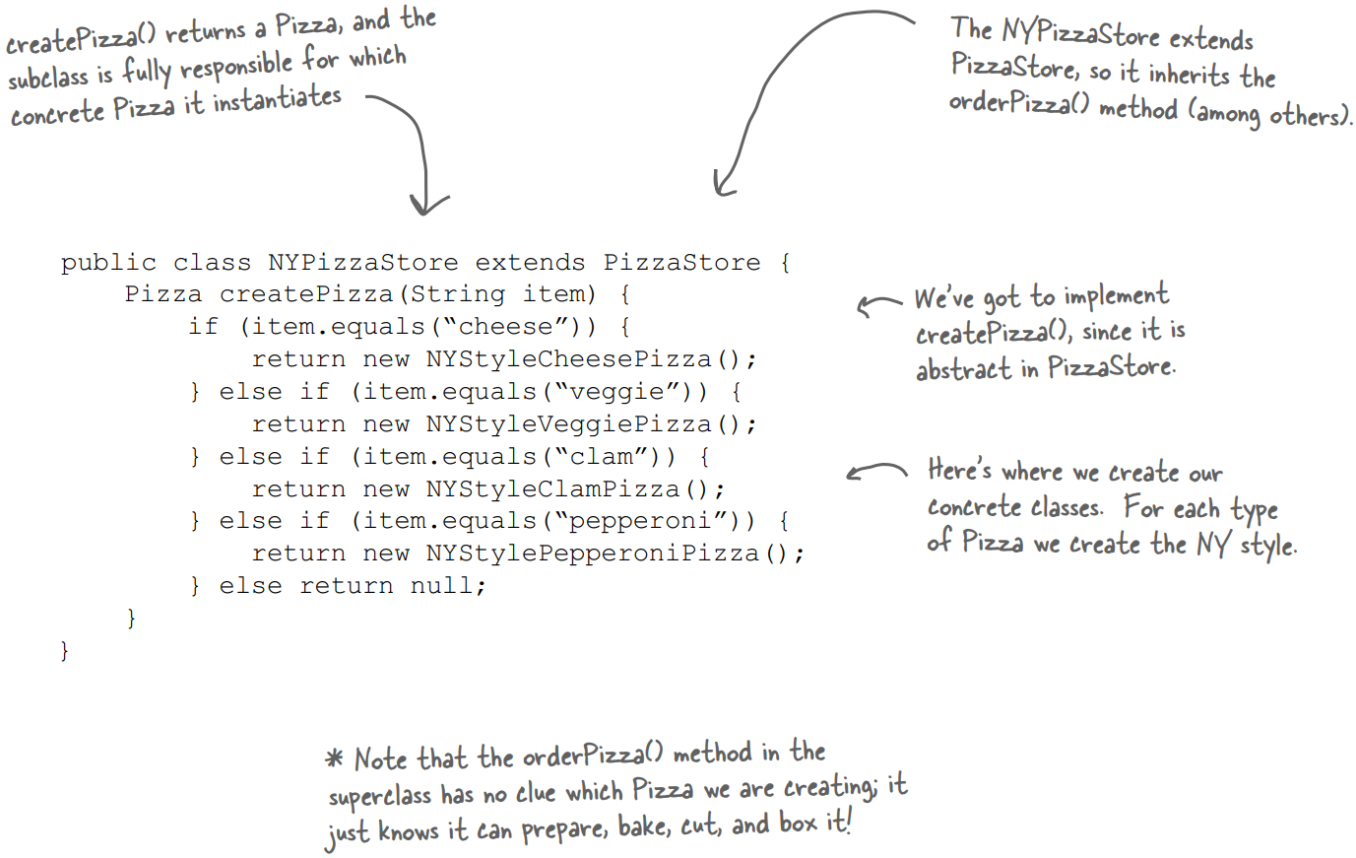
Khi **orderPizza()** gọi **createdPizza()**, một trong các lớp con của bạn sẽ được gọi hành động để tạo ra một chiếc bánh pizza. Những loại pizza nào sẽ được làm? Chà, điều đó quyết định bởi sự lựa chọn cửa hàng pizza mà bạn đặt hàng từ **NYStylePizzaStore** hoặc **ChicagoStylePizzaStore**.

Vì vậy, có một quyết định mà các lớp con đưa ra trong thời gian thực (runtime) đúng không? Không, nhưng theo quan điểm của **orderPizza()**, nếu bạn chọn **NYStylePizzaStore**, lớp con đó sẽ xác định loại pizza nào được làm. Vì vậy, các lớp con không phải là người quyết định thực sự – mà đó là bạn, bạn quyết định bằng cách chọn cửa hàng nào bạn muốn – nhưng lớp con lại xác định loại pizza nào được làm.

Hãy tạo một PizzaStore

Là một nơi được nhượng quyền sẽ có lợi ích của nó. Bạn nhận được tất cả các chức năng của lớp **PizzaStore** miễn phí. Tất cả các cửa hàng địa phương cần làm là phân lớp **PizzaStore** và cung cấp phương thức **createPizza()** tạo các loại Pizza của họ. Chúng ta sẽ quan tâm ba phong cách pizza lớn cho những người được nhượng quyền.

Ở đây, phong cách khu vực New York:



Lưu ý rằng phương thức **orderPizza()** trong lớp cha không biết loại Pizza nào chúng ta đang tạo; Nó chỉ biết nó có thể chuẩn bị, nướng, cắt, và đóng hộp!

Một khi chúng ta đã xây dựng các lớp con **PizzaStore**, đã đến lúc xem về việc đặt mua một hoặc hai chiếc bánh pizza. Nhưng trước khi chúng ta làm điều đó, tại sao bạn không tự xây dựng các cửa hàng pizza **Chicago Style** và **California Style**? Lấy giấy ra và làm đi nào.

Mô tả một phương thức factory

Chỉ với một vài biến đổi cho **PizzaStore**, chúng tôi đã chuyển từ việc chỉ có một đối tượng xử lý việc khởi tạo các lớp cụ thể của chúng tôi thành một tập hợp các lớp con hiện đang đảm nhận trách nhiệm đó. Hãy để xem xét kỹ hơn:

```
public abstract class PizzaStore {

    public Pizza orderPizza(String type) {
        Pizza pizza;

        pizza = createPizza(type);

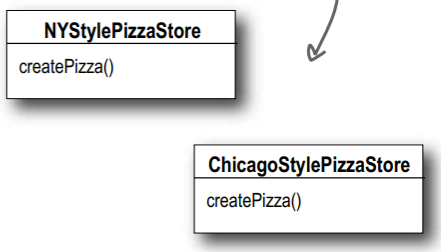
        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();

        return pizza;
    }

    protected abstract Pizza createPizza(String type);

    // other methods here
}
```

The subclasses of PizzaStore handle object instantiation for us in the createPizza() method.



All the responsibility for instantiating Pizzas has been moved into a **method** that acts as a **factory**.

Code Up Close

Một factory method xử lý việc tạo đối tượng và gói nó trong một lớp con. Điều này tách client code trong superclass khỏi code tạo object dưới subclass.

A factory method may be parameterized (or not) to select among several variations of a product.

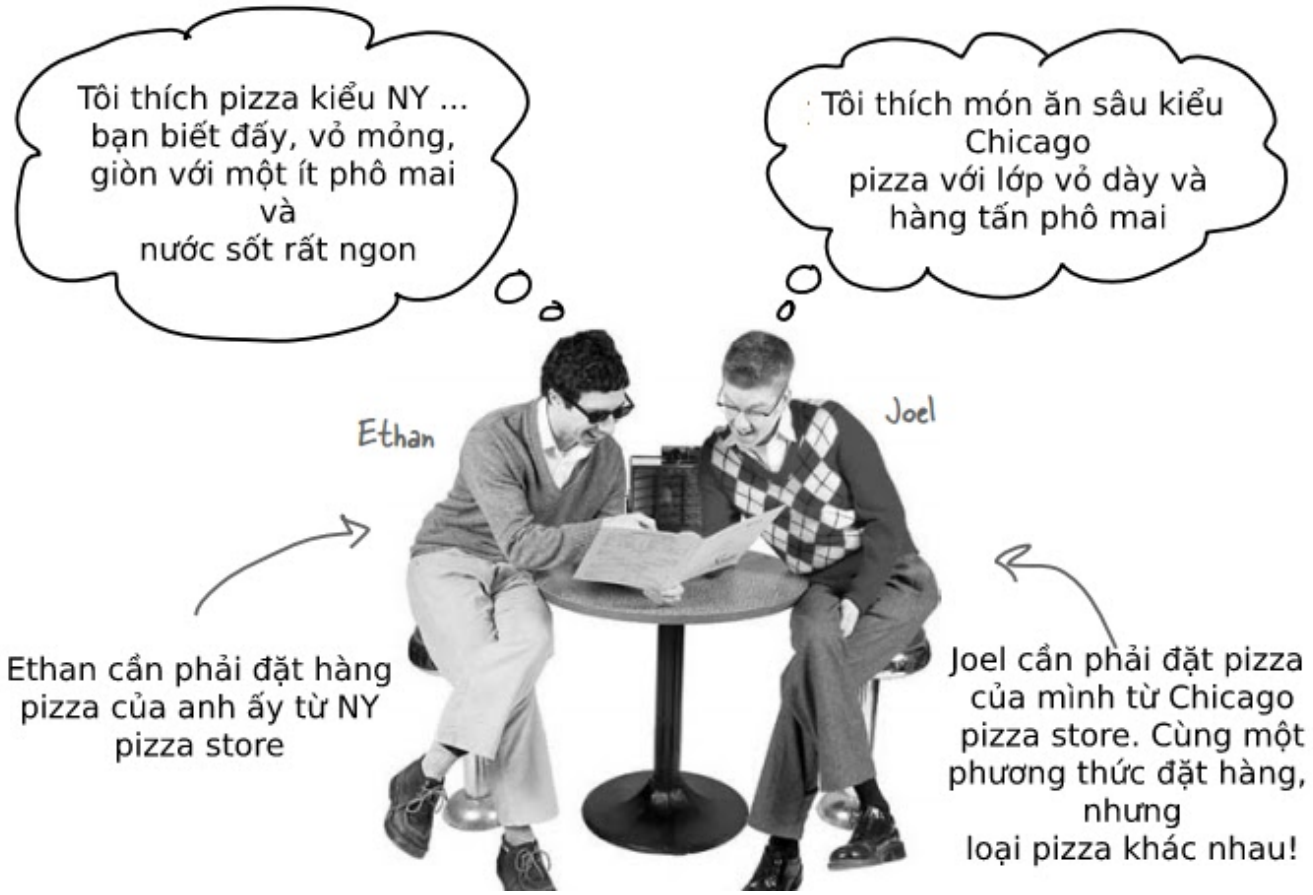
```
abstract Product factoryMethod(String type)
```

A factory method is abstract so the subclasses are counted on to handle object creation.

A factory method returns a Product that is typically used within methods defined in the superclass.

A factory method isolates the client (the code in the superclass, like orderPizza()) from knowing what kind of concrete Product is actually created.

HÃY CÙNG XEM CÁCH THỨC HOẠT ĐỘNG CỦA NÓ: ĐẶT MỘT CÁI PIZZA VỚI FACTORY METHOD



VẬY LÀM THẾ NÀO ĐỂ HỌ ĐẶT HÀNG?

1. Đầu tiên, Joel và Ethan cần một thể hiện (instance) của PizzaStore. Joel thì cần khởi tạo một ChicagoPizzaStore và Ethan cần một NYPizzaStore.
2. Với một PizzaStore trong tay, cả Ethan và Joel đều gọi phương thức orderPizza() và điền type pizza mà họ muốn (phô mai, rau, v.v.).
3. Để tạo ra các loại pizza, phương thức createPizza() được gọi, được định nghĩa trong hai lớp con NYPizzaStore và ChicagoPizzaStore. Như chúng tôi đã định nghĩa chúng, NYPizzaStore tạo ra một chiếc bánh pizza theo phong cách NY và ChicagoPizzaStore tạo ra chiếc bánh pizza kiểu Chicago. Trong cả hai trường hợp, Pizza được trả về phương thức orderPizza().
4. Phương thức orderPizza() không biết loại pizza nào được tạo ra, nhưng nó biết nó là một loại pizza và nó chuẩn bị, làm bánh, cắt, và đóng hộp cho Ethan và Joel.

HÃY CÙNG KIỂM TRA XEM NHỮNG CHIẾC PIZZA NÀY THỰC SỰ ĐƯỢC SẢN XUẤT NHƯ THẾ NÀO KHI ORDER...

1. Hãy để theo dõi đơn hàng của Ethan, trước tiên chúng ta cần một NY PizzaStore:

```
PizzaStore nyPizzaStore = new NYPizzaStore();
```

Creates a instance of NYPizzaStore.



2. Bây giờ chúng ta có một cửa hàng, chúng ta có thể đặt hàng:

```
nyPizzaStore.orderPizza("cheese");
```

The orderPizza() method is called on the nyPizzaStore instance (the method defined inside PizzaStore runs).

createPizza("cheese")

3. Phương thức orderPizza() sau đó sẽ gọi phương thức createPizza():

```
Pizza pizza  = createPizza("cheese");
```

Remember, createPizza(), the factory method, is implemented in the subclass. In this case it returns a NY Cheese Pizza.



4. Cuối cùng, chúng ta có chiếc bánh pizza chưa chuẩn bị trong tay và phương thức orderPizza() kết thúc việc chuẩn bị nó:

```
pizza.prepare();
pizza.bake();
pizza.cut();
pizza.box();
```

All of these methods are defined in the specific pizza returned from the factory method createPizza(), defined in the NYPizzaStore.

The orderPizza() method gets back a Pizza, without knowing exactly what concrete class it is.

Chúng ta vừa quên một thứ: TẠO PIZZA!

PIZZASTORE CỦA CHÚNG TA SẼ KHÔNG PHỔ BIẾN NẾU KHÔNG CÓ MỘT SỐ LOẠI PIZZA, VÌ VẬY HÃY LÀM CHÚNG:

We'll start with an abstract Pizza class and all the concrete pizzas will derive from this.

```
public abstract class Pizza {
    String name;
    String dough;
    String sauce;
    ArrayList toppings = new ArrayList();

    void prepare() {
        System.out.println("Preparing " + name);
        System.out.println("Tossing dough...");
        System.out.println("Adding sauce...");
        System.out.println("Adding toppings: ");
        for (int i = 0; i < toppings.size(); i++) {
            System.out.println("   " + toppings.get(i));
        }
    }

    void bake() {
        System.out.println("Bake for 25 minutes at 350");
    }

    void cut() {
        System.out.println("Cutting the pizza into diagonal slices");
    }

    void box() {
        System.out.println("Place pizza in official PizzaStore box");
    }

    public String getName() {
        return name;
    }
}
```

Each Pizza has a name, a type of dough, a type of sauce, and a set of toppings.

The abstract class provides some basic defaults for baking, cutting and boxing.

Preparation follows a number of steps in a particular sequence.

REMEMBER: we don't provide import and package statements in the code listings. Get the complete source code from the headfirstlabs web site. You'll find the URL on page xxxiii in the Intro.

BÂY GIỜ CHÚNG TA CHỈ CẦN MỘT SỐ LỚP CON CỤ THỂ ... LÀM THẾ NÀO ĐỂ XÁC ĐỊNH CHEESE PIZZAS KIỂU NEW YORK HAY CHICAGO?

```
public class NYStyleCheesePizza extends Pizza {
    public NYStyleCheesePizza() {
        name = "NY Style Sauce and Cheese Pizza";
        dough = "Thin Crust Dough";
        sauce = "Marinara Sauce";

        toppings.add("Grated Reggiano Cheese");
    }
}
```

The NY Pizza has its own marinara style sauce and thin crust.

And one topping, reggiano cheese!

```
public class ChicagoStyleCheesePizza extends Pizza {
    public ChicagoStyleCheesePizza() {
        name = "Chicago Style Deep Dish Cheese Pizza";
        dough = "Extra Thick Crust Dough";
        sauce = "Plum Tomato Sauce";

        toppings.add("Shredded Mozzarella Cheese");
    }

    void cut() {
        System.out.println("Cutting the pizza into square slices");
    }
}
```

The Chicago Pizza uses plum tomatoes as a sauce along with extra thick crust.

The Chicago style deep dish pizza has lots of mozzarella cheese!

The Chicago style pizza also overrides the cut() method so that the pieces are cut into squares.

Bạn đã đợi đủ lâu, thời gian này cho một vài cái pizza!

```
public class PizzaTestDrive {

    public static void main(String[] args) {
        PizzaStore nyStore = new NYPizzaStore();
        PizzaStore chicagoStore = new ChicagoPizzaStore();

        Pizza pizza = nyStore.orderPizza("cheese");
        System.out.println("Ethan ordered a " + pizza.getName() + "\n");

        pizza = chicagoStore.orderPizza("cheese");
        System.out.println("Joel ordered a " + pizza.getName() + "\n");
    }
}
```

First we create two different stores.

Then use one one store to make Ethan's order.

And the other for Joel's.

```
File Edit Window Help YouWantMootzOnThatPizza?
%java PizzaTestDrive

Preparing NY Style Sauce and Cheese Pizza
Tossing dough...
Adding sauce...
Adding toppings:
   Grated Regiano cheese
Bake for 25 minutes at 350
Cutting the pizza into diagonal slices
Place pizza in official PizzaStore box
Ethan ordered a NY Style Sauce and Cheese Pizza

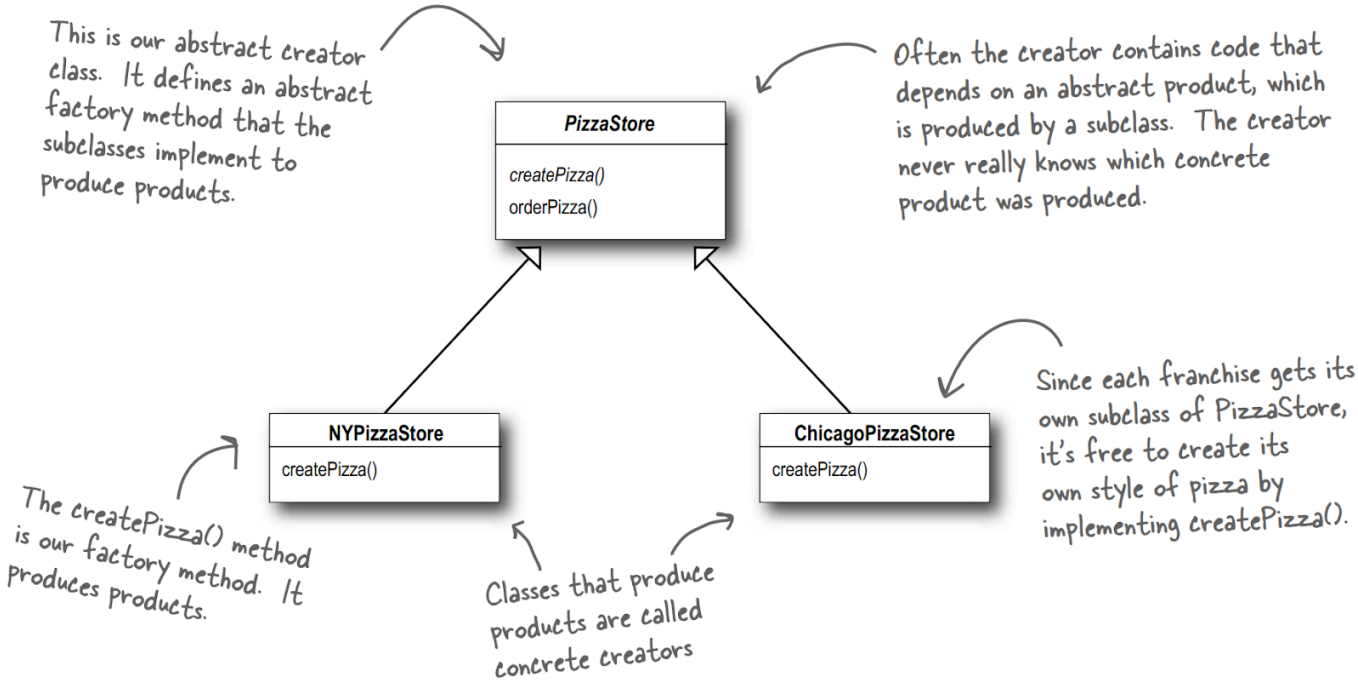
Preparing Chicago Style Deep Dish Cheese Pizza
Tossing dough...
Adding sauce...
Adding toppings:
   Shredded Mozzarella Cheese
Bake for 25 minutes at 350
Cutting the pizza into square slices
Place pizza in official PizzaStore box
Joel ordered a Chicago Style Deep Dish Cheese Pizza
```

Both pizzas get prepared, the toppings added, and the pizzas baked, cut and boxed. Our superclass never had to know the details, the subclass handled all that just by instantiating the right pizza.

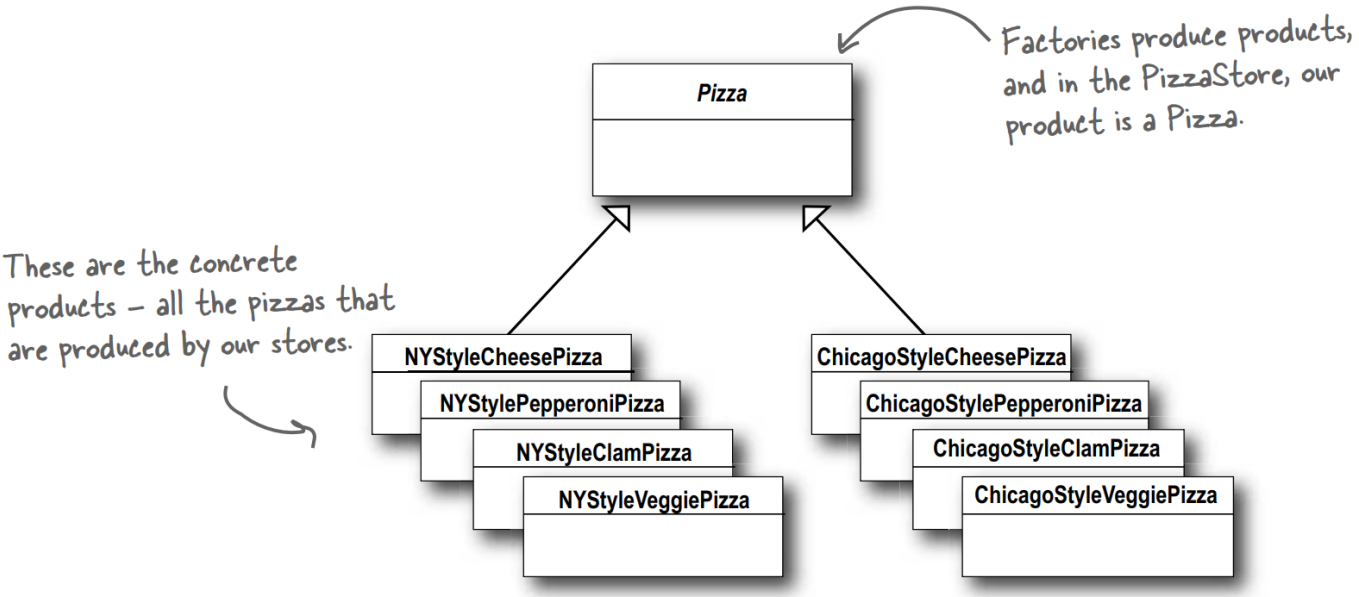
Cuối cùng cũng đến lúc biết về Factory Pattern

Tất cả các mẫu factory đóng gói việc tạo đối tượng. Mẫu **Factory Pattern** đóng gói việc tạo đối tượng bằng cách để các lớp con quyết định những đối tượng sẽ tạo. Hãy cùng kiểm tra các sơ đồ lớp này để xem ai là "người chơi chính" trong mẫu này:

LỚP CREATOR



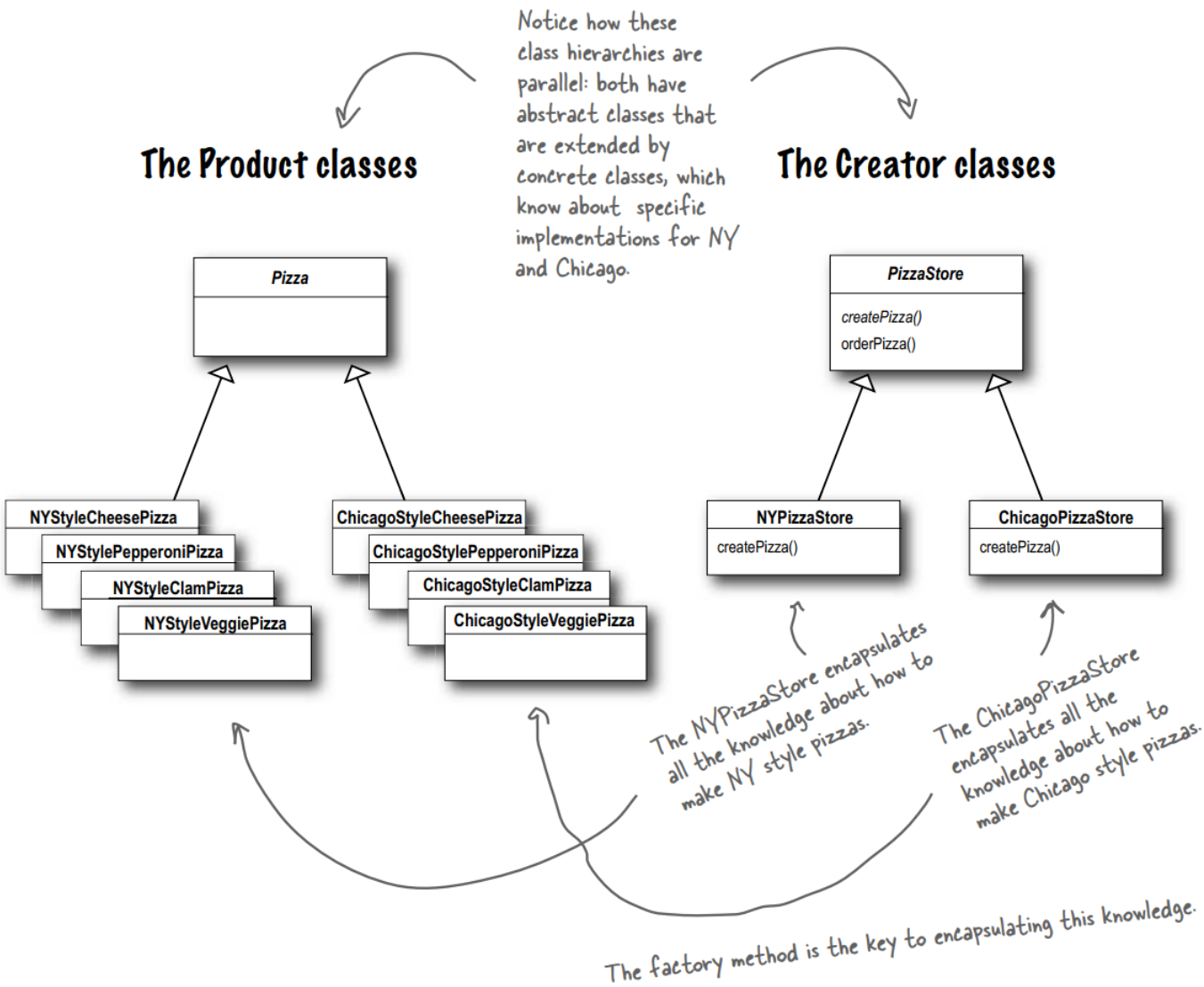
LỚP PRODUCT



Một góc nhìn khác của Factory Pattern: hệ thống phân cấp lớp dạng song song

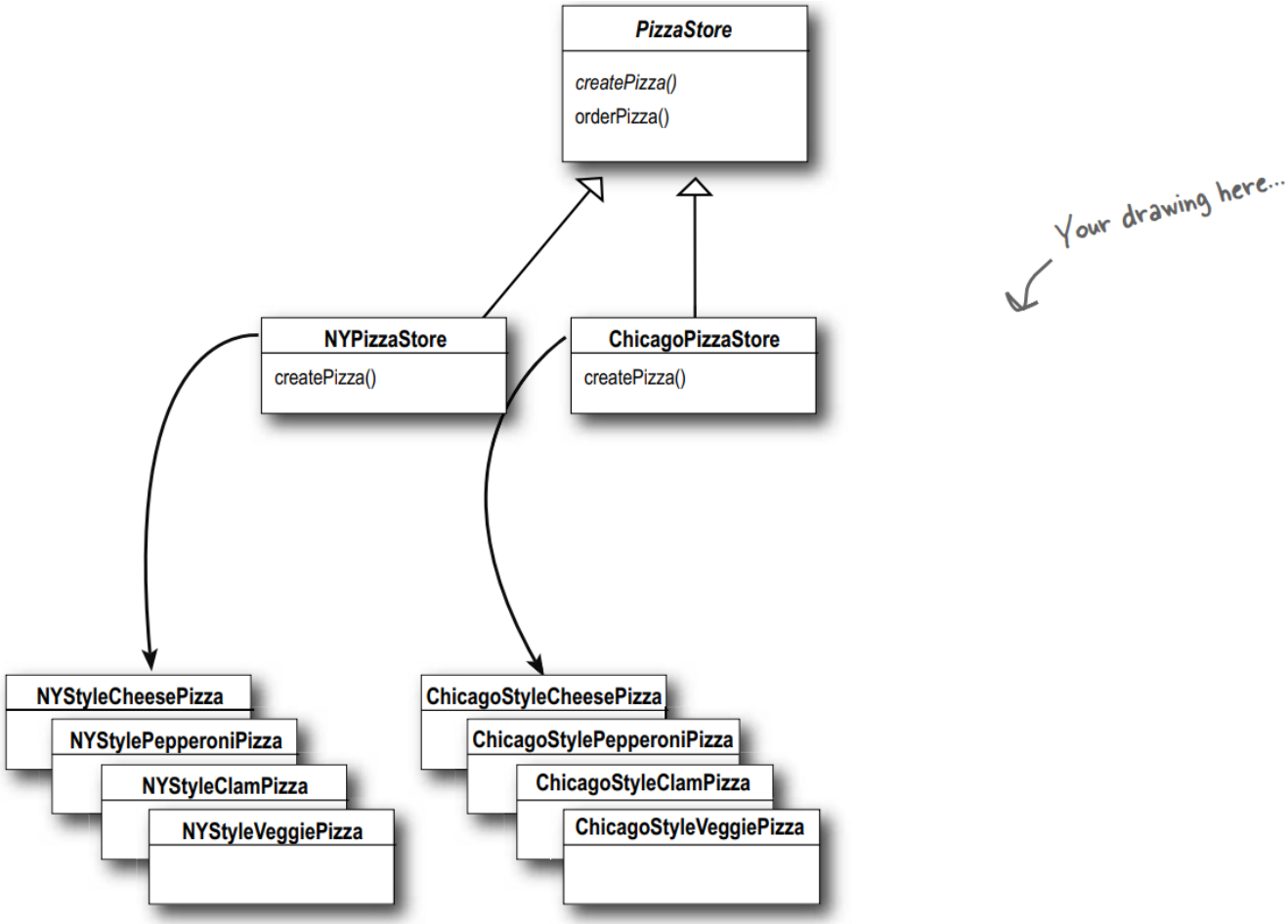
Chúng tôi đã thấy rằng factory pattern cung cấp một framework bằng cách cung cấp một phương thức **orderPizza()** được kết hợp với một factory method. Một cách khác để xem mẫu này như một framework là trong cách nó gói gọn "thông tin về sản phẩm" vào mỗi creator.

Hãy cùng xem xét hai hệ thống phân cấp lớp đặt song song và xem chúng liên quan như thế nào:



Thiết kế

Chúng ta cần một loại bánh pizza khác cho những người California. Vẽ một tập hợp các lớp song song khác mà bạn cần để thêm một vùng California mới vào chuỗi cửa hàng **PizzaStore** của chúng tôi.



Được rồi, bây giờ viết năm thứ kỳ lạ nhất mà bạn có thể nghĩ ra để đặt trên một chiếc bánh pizza.

Sau đó, bạn sẽ sẵn sàng kinh doanh làm pizza ở California!

Định nghĩa Factory Pattern

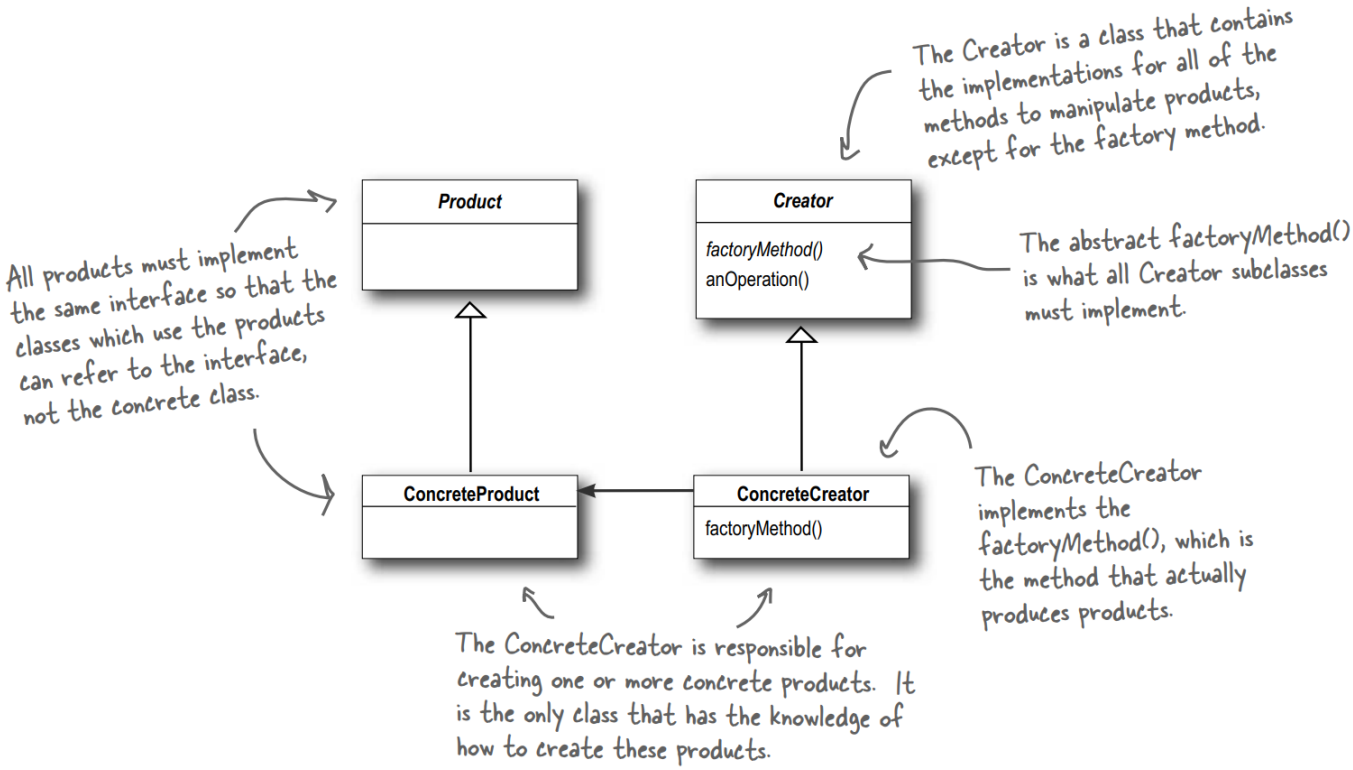
Đây là thời gian để đưa ra định nghĩa chính thức của **Factory Pattern** (hay **Factory Method Pattern**):

> **The Factory Method Pattern** defines an interface for creating an object, but lets subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.

(Factory Pattern xác định một interface để tạo một đối tượng, nhưng cho phép các lớp con quyết định lớp nào sẽ khởi tạo. Factory Pattern giao việc khởi tạo một đối tượng cụ thể cho lớp con)

Như với mọi nhà máy, Mẫu **Factory Pattern** cho chúng ta một cách để gói gọn các khởi tạo của các loại cụ thể. Nhìn vào sơ đồ lớp bên dưới, bạn có thể thấy rằng abstract **Creator** cung cấp cho bạn một interface với phương thức khởi tạo các đối tượng, còn được gọi là "factory method". Bất kỳ lớp nào khác implement lớp abstract **Creator** đều được viết để vận hành trên các sản phẩm (object) được tạo bởi factory method. Chỉ các lớp con thực sự thực hiện phương thức factory và tạo ra sản phẩm (object).

Như trong định nghĩa chính thức, bạn sẽ nghe các developer nói rằng **Factory Pattern** cho phép các lớp con quyết định lớp nào sẽ khởi tạo. Họ nói rằng "quyết định" không phải vì mô hình cho phép chính các lớp con quyết định trong runtime, mà bởi vì lớp creator được viết mà không cần biết về các sản phẩm thực tế sẽ được tạo, được quyết định hoàn toàn bởi sự lựa chọn của lớp con.



Không có câu hỏi ngớ ngẩn

Hỏi: Có lợi ích gì không nếu Factory Pattern chỉ có một ConcreteCreator?

Trả lời: Factory Pattern rất hữu ích nếu bạn chỉ có một concrete creator vì bạn đang tách riêng việc triển khai sản phẩm khỏi việc sử dụng nó. Nếu bạn thêm sản phẩm bổ sung hoặc thay đổi một Sản phẩm triển khai, nó sẽ không ảnh hưởng đến Creator của bạn (vì Creator không được liên kết chặt chẽ với bất kỳ ConcreteProduct nào).

Hỏi: Có đúng không khi nói rằng các cửa hàng NY và Chicago của chúng tôi được triển khai bằng **Simple Factory**? Chúng trông giống như nó.

Trả lời: Chúng giống nhau, nhưng được sử dụng theo những cách khác nhau. Mặc dù việc triển khai của mỗi concrete store trông rất giống **SimplePizzaFactory**, hãy nhớ rằng các concrete store đang extends một lớp đã định nghĩa **createPizza()** là một phương thức trừu tượng. Tùy thuộc vào mỗi cửa hàng để xác định hành vi của phương thức **createPizza()**. Trong Simple Factory, factory là một đối tượng khác được kết hợp với **PizzaStore**.

Hỏi: Có phải factory method và **Creator** luôn trừu tượng?

Trả lời: Không, bạn có thể xác định factory method mặc định để sản xuất một số sản phẩm cụ thể. Khi đó, bạn luôn có phương tiện tạo sản phẩm ngay cả khi không có lớp con của Creator.

Hỏi: Mỗi cửa hàng có thể tạo ra bốn loại pizza khác nhau dựa trên type được truyền vào. Có phải tất cả các creator con đều tạo ra nhiều loại sản phẩm, hay đôi khi chúng chỉ làm một loại?

Trả lời: Chúng tôi đã implemented những gì được biết thành tham số cho factory method. Nó có thể tạo nhiều hơn một đối tượng dựa trên tham số được truyền vào, như bạn thấy. Tuy nhiên, thông thường, một nhà máy chỉ tạo một đối tượng và không có tham số truyền vào. Cả hai đều là cách viết hợp lệ của mẫu.

Hỏi: Các loại tham số của bạn dường như không an toàn. Tôi chỉ truyền một chuỗi String! Điều gì sẽ xảy ra nếu tôi yêu cầu một món "**CalmPizza**" (ý nói truyền tùy ý một chuỗi String nào cũng được)?

Trả lời: Bạn chắc chắn là đúng và điều đó là nguyên nhân xảy ra lỗi thời gian chạy (runtime error). Có một vài kỹ thuật có thể áp dụng để tạo một tham số an toàn hơn hoặc một cách nói khác, đảm bảo lỗi trong các parameter có thể được catch (bắt) tại thời điểm biên dịch. Chẳng hạn, bạn có thể tạo các đối tượng đại diện cho các loại tham số, sử dụng các hằng số hoặc trong Java 5, bạn có thể sử dụng ***Enums***.

Hỏi: Tôi vẫn còn một chút bối rối về sự khác biệt giữa **Simple Factory** và **Factory Pattern**. Chúng trông rất giống nhau, ngoại trừ trong **Factory Pattern**, lớp returns pizza là một lớp con. Bạn có thể giải thích không?

Trả lời: Bạn đúng về điều đó, các lớp con trông rất giống Simple Factory, tuy nhiên, suy nghĩ của Simple Factory như một "shot deal", trong khi với Factory Pattern, bạn đang tạo một framework để cho phép các lớp con quyết định việc triển khai nào sẽ được sử dụng. Ví dụ, phương thức ***orderPizza()*** trong Factory Pattern cung cấp một framework chung để tạo ra các loại pizza (dựa trên phương thức factory) khi đó các lớp con sẽ trực tiếp tạo ra những chiếc pizza. Bằng cách phân lớp **PizzaStore**, bạn quyết định sản phẩm

cụ thể nào ***orderPizza()*** sẽ return. So sánh với **SimpleFactory**, cung cấp cho bạn một cách để đóng gói việc tạo đối tượng, nhưng không cho phép bạn linh hoạt như Factory Pattern vì không có cách nào để thay đổi các sản phẩm bạn tạo ra.

Sư phụ và Học trò…

Sư phụ: Grasshopper, cho thầy biết con đã học được những gì?

Học trò: Sư phụ, con đã thực hiện nghiên cứu của mình về "đóng gói những gì thay đổi" về phạm vi.

Sư phụ: Tiếp tục đi…

Học trò: Con đã học được rằng người ta có thể đóng gói code tạo ra các đối tượng. Khi có code khởi tạo các lớp cụ thể, đây là một khu vực thường xuyên thay đổi. Con đã học được một kỹ thuật gọi là "factory pattern", cho phép con gói gọn hành vi khởi tạo này.

Sư phụ: Và những "factory" đó, chúng có lợi ích gì?

Học trò: Có rất nhiều. Bằng cách đặt tất cả code tạo đối tượng của mình vào một object hoặc phương thức, con tránh trùng lặp trong code của mình và cung cấp một nơi để thực hiện việc bảo trì. Điều đó cũng có nghĩa là các client chỉ phụ thuộc vào các interface thay vì các lớp cụ thể cần thiết để khởi tạo các đối tượng. Như con đã học được trong các nghiên cứu của mình, điều này cho phép con lập trình trên một giao diện (program to an interface), không phải là lập trình trên implementation và điều đó làm cho code của con linh hoạt hơn và có thể mở rộng trong tương lai.

Sư phụ: Vâng Grasshopper, bản năng OO của con đang phát triển. Con có bất kỳ câu hỏi cho sư phụ ngày hôm nay không?

Học trò: Sư phụ, con biết rằng bằng cách đóng gói việc tạo đối tượng, con đang "coding to abstractions" (code đến sự trừu tượng) và tách code client của con khỏi các triển khai thực tế. Nhưng factory code vẫn phải sử dụng các lớp concrete (lớp con kế thừa factory) để khởi tạo các đối tượng thực. Đây có phải là một cú lừa?

Sư phụ: Grasshopper, Khởi tạo đối tượng là một thực tế của cuộc sống; chúng ta phải tạo các đối tượng hoặc chúng ta sẽ không bao giờ tạo một chương trình Java. Nhưng, với kiến thức về thực tế này, chúng ta có thể thiết kế code của mình, vì vậy chúng ta đã điều chỉnh creation code này giống như con cừu mà len của nó sẽ kéo qua mắt con *(nguyên văn: whose wool you would pull over your eyes – thành ngữ chỉ sự lừa ai đó)*. Sau khi được điều chỉnh, chúng ta có thể bảo vệ và bảo trì code khởi tạo (creation code). Nếu chúng ta để creation code của mình chạy tự do, thì chúng ta sẽ không bao giờ thu thập được "len" của nó.

Học trò: Sư phụ, con thấy sự thật trong này.

Sư phụ: Như thầy đã biết con sẽ làm. Bây giờ, đi và suy nghĩ về các phụ thuộc đối tượng (object dependencies).

Cửa hàng Pizza "rất phụ thuộc"

Hãy giả vờ như bạn đã từng nghe nói về **Factory Pattern**. Ở đây, một phiên bản của PizzaStore không sử dụng factory pattern; hãy đếm số đối tượng con tạo pizza lớp này phụ thuộc. Nếu bạn đã thêm pizza kiểu California vào PizzaStore này, thì có bao nhiêu đối tượng sẽ phụ thuộc vào đó?

```
public class DependentPizzaStore {

    public Pizza createPizza(String style, String type) {
        Pizza pizza = null;
        if (style.equals("NY")) {
            if (type.equals("cheese")) {
                pizza = new NYStyleCheesePizza();
            } else if (type.equals("veggie")) {
                pizza = new NYStyleVeggiePizza();
            } else if (type.equals("clam")) {
                pizza = new NYStyleClamPizza();
            } else if (type.equals("pepperoni")) {
                pizza = new NYStylePepperoniPizza();
            }
        } else if (style.equals("Chicago")) {
            if (type.equals("cheese")) {
                pizza = new ChicagoStyleCheesePizza();
            } else if (type.equals("veggie")) {
                pizza = new ChicagoStyleVeggiePizza();
            } else if (type.equals("clam")) {
                pizza = new ChicagoStyleClamPizza();
            } else if (type.equals("pepperoni")) {
                pizza = new ChicagoStylePepperoniPizza();
            }
        } else {
            System.out.println("Error: invalid type of pizza");
            return null;
        }
        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();
        return pizza;
    }
}
```

Handles all the NY style pizzas

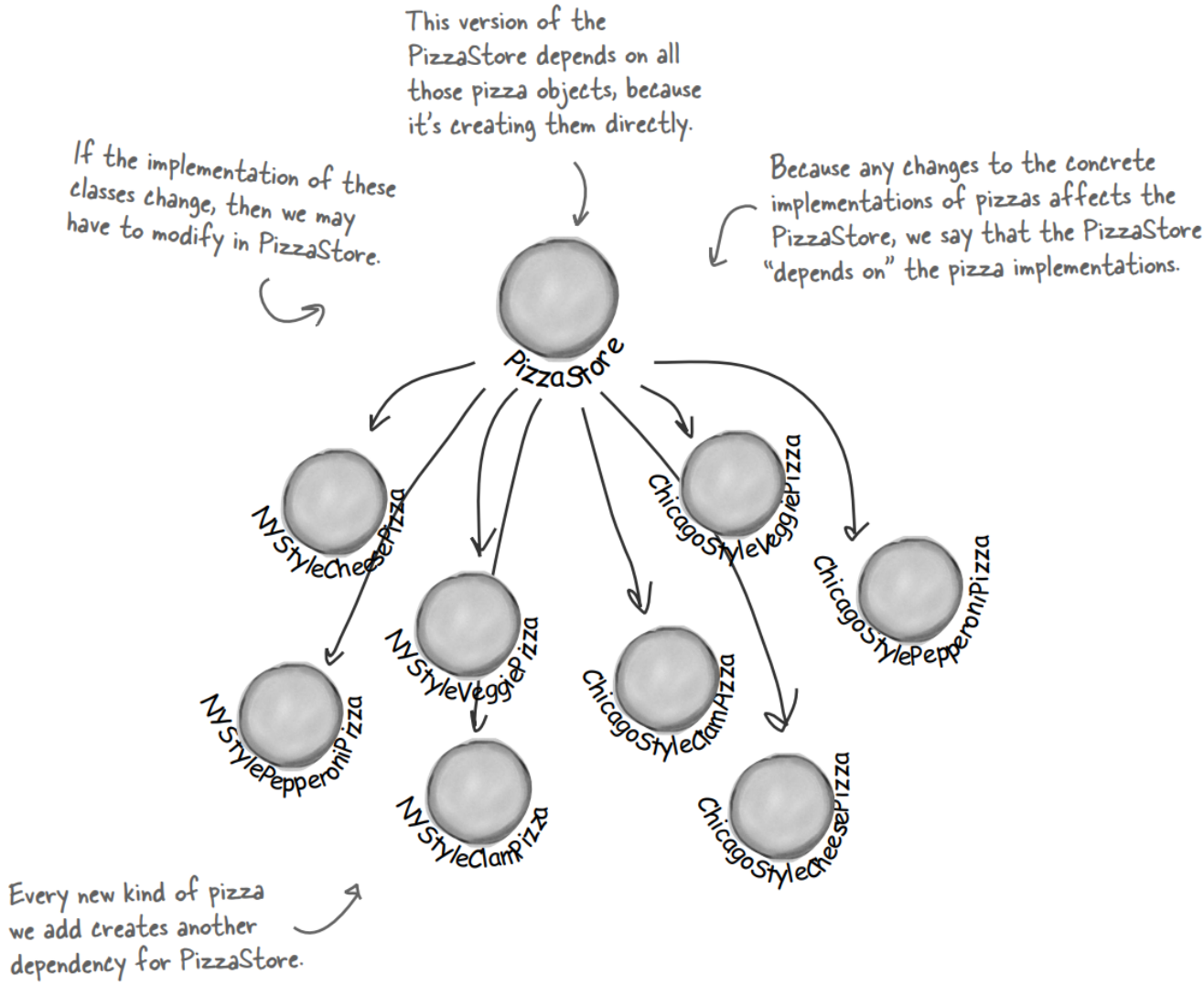
Handles all the Chicago style pizzas

You can write your answers here: _____ number _____ number with California too

Nhìn vào sự phụ thuộc đối tượng

Khi bạn trực tiếp khởi tạo một đối tượng, bạn phụ thuộc vào lớp cụ thể của nó. Hãy xem lại **PizzaStore** rất nhiều sự phụ thuộc của chúng tôi. Nó tạo ra tất cả các đối tượng pizza ngay trong lớp **PizzaStore** thay vì ủy thác cho một lớp **Factory**.

Nếu chúng ta vẽ một sơ đồ biểu thị phiên bản đó của PizzaStore và tất cả các đối tượng mà nó phụ thuộc vào, thì ở đây, nó trông như thế nào:



Nguyên tắc nghịch đảo phụ thuộc

Cần phải rõ ràng rằng việc giảm sự phụ thuộc vào các lớp cụ thể trong code của chúng ta là một điều tốt. Thực tế, chúng ta đã có một nguyên tắc thiết kế OO chính thức hóa khái niệm này; nó thậm chí còn có một tên chính thức lớn: Nguyên tắc nghịch đảo phụ thuộc (*Dependency Inversion Principle*).

Ở đây, nguyên tắc chung:



Design Principle

Depend upon abstractions. Do not depend upon concrete classes.

(Kiểu trừu tượng không nên phụ thuộc kiểu chi tiết. Ngược lại, kiểu chi tiết nên phụ thuộc kiểu trừu tượng)

Lúc đầu, nguyên tắc này nghe có vẻ giống lập trình trên một giao diện, không phải là triển khai ("Program to an interface, not an implementation"), phải không? Nó là tương tự; tuy nhiên, Nguyên tắc nghịch đảo phụ thuộc đưa ra một tuyên bố thậm chí còn mạnh mẽ hơn về sự trừu tượng. Nó cho thấy rằng các thành phần cấp cao của chúng ta không nên phụ thuộc vào các thành phần cấp thấp; thay vào đó, cả hai nên phụ thuộc vào trừu tượng.

(*Chú thích thêm: Một thành phần cấp cao là một lớp có hành vi được xác định theo các khía cạnh của các thành phần ở cấp độ thấp.*

Ví dụ: PizzaStore là một thành phần cấp cao vì hành vi của nó được định nghĩa dưới dạng pizza – nó tạo ra tất cả các đối tượng pizza khác nhau, chuẩn bị, nướng, cắt và đóng hộp chúng, trong khi các loại pizza mà nó sử dụng là các thành phần cấp thấp.)

Nhưng cái quái đó có nghĩa là gì?

Chà, hãy để bắt đầu bằng cách nhìn lại sơ đồ cửa hàng pizza trên trang trước. PizzaStore là thành phần cấp cao của chúng ta, và việc triển khai pizza là các thành phần cấp thấp, và rõ ràng PizzaStore phụ thuộc vào các lớp pizza cụ thể.

Bây giờ, nguyên tắc này cho chúng ta biết thay vào đó chúng ta nên viết code để chúng ta phụ thuộc vào trừu tượng, chứ không phải các lớp cụ thể. Điều đó áp dụng cho cả các mô-đun cấp cao và mô-đun cấp thấp của chúng ta.

Nhưng làm thế nào để chúng ta làm điều này? Hãy nghĩ về cách áp dụng nguyên tắc này cho việc triển khai PizzaStore rất phụ thuộc của chúng ta…

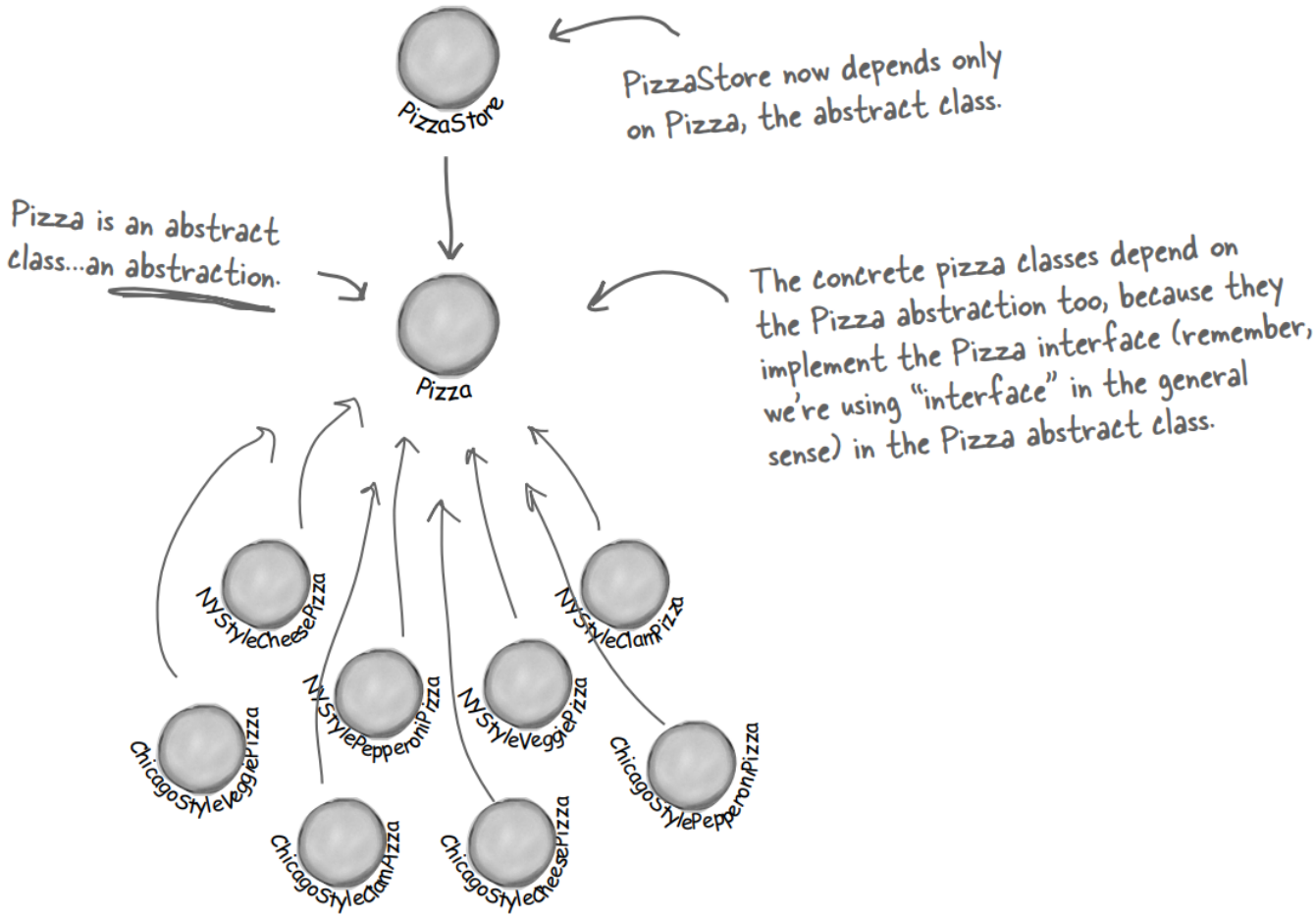
Áp dụng nguyên tắc

Bây giờ, vấn đề chính với PizzaStore là nó phụ thuộc vào từng loại bánh pizza vì nó thực sự tạo ra trong phương thức **orderPizza()** của nó.

Mặc dù chúng ta đã tạo ra một sự trừu tượng hóa, Pizza, tuy nhiên chúng ta vẫn tạo ra những chiếc bánh pizza cụ thể trong lớp này, vì vậy chúng ta không nhận được nhiều lợi ích từ sự trừu tượng này.

Làm thế nào chúng ta có thể đưa những cài đặt đó ra khỏi phương thức **orderPizza()**? Vâng, như chúng ta đã biết, **Factory Pattern** cho phép chúng ta làm điều đó.

Vì vậy, sau khi chúng ta áp dụng **Factory Pattern**, sơ đồ của chúng ta trông như thế này:



Sau khi áp dụng **Factory Pattern**, bạn sẽ nhận thấy rằng thành phần cấp cao của chúng tôi PizzaStore và các thành phần cấp thấp của chúng tôi, những loại pizza, cả hai đều phụ thuộc vào Pizza, sự trừu tượng. **Factory Pattern** không phải là kỹ thuật duy nhất tuân thủ Nguyên tắc nghịch đảo phụ thuộc, nhưng nó là một trong những kỹ thuật mạnh.



Đâu là "sự đảo ngược" trong Dependency Inversion Principle?

"Sự đảo ngược" trong *Dependency Inversion Principle* là vì nó đảo ngược cách bạn thường nghĩ về thiết kế OO của mình. Nhìn vào sơ đồ trên trang trước, lưu ý rằng các thành phần cấp thấp hiện phụ thuộc vào mức độ trừu tượng cao hơn. Tương tự, thành phần cấp cao cũng được gắn với sự trừu tượng cao hơn. Vì vậy, biểu đồ phụ thuộc top-to-bottom mà chúng tôi đã vẽ một vài trang trở lại đã tự đảo ngược, với cả các mô-đun cấp cao và cấp thấp bây giờ tùy thuộc vào sự trừu tượng hóa.

Hãy suy nghĩ lướt qua về quá trình thiết kế điển hình và xem cách giới thiệu nguyên tắc có thể đảo ngược cách chúng ta nghĩ về thiết kế …

Đảo ngược suy nghĩ của bạn

Được rồi, vì vậy bạn cần phải thực hiện một **PizzaStore**. Ý nghĩ đầu tiên xuất hiện trong đầu bạn là gì?



Phải, bạn bắt đầu từ đầu và nghiên cứu về các lớp cụ thể, các loại pizza. Nhưng, như bạn đã thấy, bạn không muốn cửa hàng của mình quan tâm về các loại bánh concrete pizza, bởi vì sau đó nó sẽ phụ thuộc vào tất cả các lớp concrete đó!

Bây giờ, ngay lập tức, hãy "đảo ngược" suy nghĩ của bạn … thay vì bắt đầu từ các lớp concrete, hãy bắt đầu tại Pizzas và suy nghĩ về những gì bạn có thể trừu tượng.



Đúng! Bạn đang nghĩ về Pizza trừu tượng. Vì vậy, bây giờ, hãy quay lại và suy nghĩ về thiết kế của Pizza Store một lần nữa.



Nhưng để làm được điều đó, bạn sẽ phải dựa vào một nhà máy Factory để đưa concrete classes đó ra khỏi Pizza Store. Một khi bạn đã làm điều đó, các loại concrete pizza khác nhau của bạn chỉ phụ thuộc vào sự trừu tượng và cửa hàng của bạn cũng vậy. Chúng tôi đã lấy một thiết kế trong đó, store phụ thuộc vào

concrete classes và đảo ngược các phụ thuộc đó (cùng với suy nghĩ của bạn).

Một vài hướng dẫn để giúp bạn làm theo Nguyên tắc này…

Các hướng dẫn sau đây có thể giúp bạn tránh các thiết kế OO vi phạm Nguyên tắc nghịch đảo phụ thuộc:

- Không có biến giữ một tham chiếu đến một lớp cụ thể. (Nếu bạn sử dụng toán tử *new*, bạn sẽ giữ một tham chiếu đến một lớp cụ thể. Sử dụng Factory pattern để có được điều này!)
- Không có lớp xuất phát từ một lớp cụ thể. (Nếu bạn xuất phát từ một lớp cụ thể, bạn phụ thuộc vào một lớp cụ thể. Xuất phát từ một sự trừu tượng, như một giao diện hoặc một lớp trừu tượng)
- Không có phương thức nào override một phương thức của bất kỳ lớp cơ sở nào của nó (Nếu bạn override một implemented method, thì lớp cơ sở của bạn không thực sự là một sự trừu tượng hóa. Các phương thức được triển khai trong lớp cơ sở có nghĩa là được chia sẻ bởi tất cả các lớp con của bạn.)

Nhưng chờ đã, không phải những hướng dẫn này là không thể làm theo sao? Nếu tôi làm theo những điều này, tôi sẽ không bao giờ có thể viết được một chương trình nào cả!

Bạn đã chính xác! Giống như nhiều nguyên tắc của chúng tôi, đây là một hướng dẫn bạn nên cố gắng thực hiện, hơn là một quy tắc phải tuân thủ mọi lúc.

Rõ ràng, mọi chương trình Java từng được viết đều vi phạm các nguyên tắc này!

Nhưng, nếu bạn tiếp thu những hướng dẫn này và đặt chúng trong suy nghĩ của bạn khi thiết kế, bạn sẽ biết khi nào bạn vi phạm nguyên tắc và bạn có lý do chính đáng để làm như vậy. Chẳng hạn, nếu bạn có một lớp không có khả năng thay đổi và bạn biết điều đó, thì nó không vấn đề cho dù bạn có khởi tạo một lớp cụ thể trong code của mình. Nghĩ về nó; chúng ta khởi tạo các đối tượng **String** mọi lúc mà không cần suy nghĩ về nó.

Điều đó có vi phạm nguyên tắc không? Không. Nó không sao cả. Tại sao? Bởi vì **String** rất khó thay đổi.

Mặt khác, nếu một lớp bạn viết có khả năng thay đổi, bạn có một số kỹ thuật tốt như **Factory Method Pattern** để gói gọn sự thay đổi đó.

(còn tiếp...phần 2 (https://toihocdesignpattern.com/chuong-4-factory-pattern-va-abstract-factory-pattern-phan-2.html)).